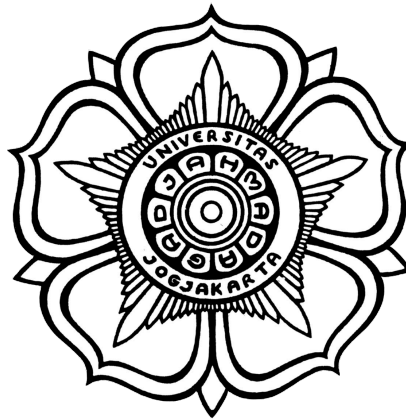


PROPOSAL TUGAS AKHIR

**ANALISIS DAN IMPLEMENTASI SISTEM *END-TO-END*
GOOD FIRST ISSUE RECOMMENDER BERBASIS MODEL PFIREC
PADA LINGKUNGAN *GITHUB OPEN-SOURCE REPOSITORY***



Andreas Notokusumo

22/493183/PA/21167

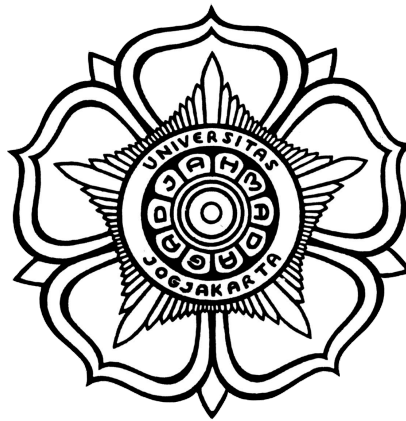
**PROGRAM STUDI S1 ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2025

PROPOSAL TUGAS AKHIR

**IMPLEMENTASI DAN ANALISIS SISTEM *END-TO-END*
GOOD FIRST ISSUE RECOMMENDER BERBASIS MODEL PFIREC
PADA LINGKUNGAN *GITHUB OPEN-SOURCE REPOSITORY***

Diajukan untuk memenuhi salah satu syarat memperoleh derajat Sarjana Ilmu
Komputer/Sarjana Sains Elektronika dan Instrumentasi



Andreas Notokusumo

22/493183/PA/21167

**PROGRAM STUDI S1 ILMU KOMPUTER
DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2025**

DAFTAR ISI

DAFTAR ISI.....	3
INTISARI.....	5
ABSTRACT.....	6
BAB I.....	1
PENDAHULUAN.....	1
1.1. Latar Belakang.....	1
1.2. Rumusan Masalah.....	3
1.3. Batasan Masalah.....	4
1.4. Tujuan Penelitian.....	5
1.5. Manfaat Penelitian.....	6
BAB II.....	7
TINJAUAN PUSTAKA.....	7
2.1. Git.....	7
2.2. GitHub.....	8
2.3. Good First Issue (GFI).....	11
2.4. Issue Recommender Berbasis AI/ML.....	12
2.4.1. Github Project Recommender.....	12
2.4.2. GiveMeLabeledIssues.....	13
2.4.3. PFIRec.....	14
2.5. Github API.....	16
2.6. Database.....	17
2.7. Pola Queue dan Background Worker.....	18
2.8. Sistem Front-End.....	20
2.9. User-Centered Design.....	21
BAB III.....	24
METODE DAN RANCANGAN PENELITIAN.....	24
3.1. Deskripsi Umum Penelitian.....	24
3.2. Metodologi.....	25
3.2.1. Tahap Perencanaan.....	26
3.2.2. Tahap Implementasi Model Machine Learning.....	27
3.2.3. Tahap Implementasi Desain UI/UX.....	28
3.2.4. Tahap Perancangan System Design.....	30
3.2.5. Tahap Implementasi Komponen Server-side.....	33
3.2.6. Tahap Implementasi Komponen Client-side.....	35
3.2.7. Tahap Pengumpulan dan Analisis Data Performa Sistem serta Perilaku Pengguna.....	36
3.2.8. Tahap Evaluasi.....	40

BAB IV.....	42
JADWAL PENELITIAN.....	42
4.1. Jadwal Penelitian.....	42
GLOSARIUM.....	44
Kontrol Versi dan Kolaborasi.....	44
Arsitektur dan Pola Pengembangan Perangkat Lunak.....	44
Desain dan Pengalaman Pengguna.....	45
Keamanan.....	45
Operasi dan Pemantauan.....	46
DAFTAR PUSTAKA.....	47

INTISARI

**IMPLEMENTASI DAN ANALISIS SISTEM *END-TO-END*
GOOD FIRST ISSUE RECOMMENDER BERBASIS MODEL
PFIREC PADA LINGKUNGAN *OPEN-SOURCE REPOSITORY***

Oleh
Andreas Notokusumo
22/493183/PA/21167

Proyek perangkat lunak yang bersifat *open-source* (*Open Source Software* atau OSS) seringkali menghadapi kendala dalam proses orientasi kontributor baru, sebuah tantangan yang menjadi fokus penelitian ini dengan mengajukan implementasi dan analisis sistem *end-to-end* (E2E) untuk rekomendasi *Good First Issue* (GFI) berbasis model *Personalized First Issue Recommender* (PFIRec). Meskipun model PFIRec menawarkan pendekatan teoritis yang kuat untuk rekomendasi *issue* yang terpersonalisasi, adopsinya di lingkungan industri masih terbatas. Proyek ini bertujuan untuk mengatasi keterbatasan tersebut dengan mengembangkan sistem E2E yang terstruktur dengan memperhatikan arsitektur *scalable* yang sesuai standar industri dan antarmuka dengan pendekatan *user-centric design*. Komponen-komponen kunci akan mencakup layanan konstruksi dataset, layanan inferensi model yang memanfaatkan model PFIRec, dan layanan antarmuka pengguna. Untuk mengelola aliran data dan tugas-tugas asinkron secara efisien, sistem akan memanfaatkan teknologi seperti *service worker* dan *message queue*. Adapun, implementasi E2E ini sangat penting untuk menjembatani kerangka kerja teoritis PFIRec dengan solusi yang praktis, berkelanjutan, dan berpusat pada pengguna demi meningkatkan keterlibatan kontributor dalam ekosistem OSS di Github.

Kata Kunci:

End-to-End System, Microservices, Queue, Service Worker, Open-Source Software, Git, Github, Good First Issue, Recommender System, PFIRec

ABSTRACT

**IMPLEMENTATION AND ANALYSIS OF AN END-TO-END
GOOD FIRST ISSUE RECOMMENDER SYSTEM BASED ON
THE PFIREC MODEL IN AN OPEN-SOURCE REPOSITORY
ENVIRONMENT**

by
Andreas Notokusumo
22/493183/PA/21167

Open-source software (OSS) projects frequently encounter challenges in the process of onboarding new contributors. This challenge constitutes the focus of the present research, which proposes the implementation and analysis of an end-to-end (E2E) system for Good First Issue (GFI) recommendation based on the Personalized First Issue Recommender (PFIRec) model. While the PFIRec model offers a strong theoretical approach for personalized issue recommendation, its industrial adoption is still limited. This project aims to address these limitations by developing a structured E2E system with attention to a scalable architecture compliant with industry standards and a user interface employing a user-centric design approach. Key components will include a dataset construction service, a model inference service utilizing the PFIRec model, and a user interface service. To efficiently manage data flow and asynchronous tasks, the system will leverage technologies such as service workers and message queues. Furthermore, this E2E implementation is crucial for bridging the theoretical framework of PFIRec with a practical, sustainable, and user-centric solution aimed at enhancing contributor engagement within the OSS ecosystem in Github.

Keywords:

End-to-End System, Microservices, Queue, Service Worker, Open-Source Software, Git, Github, Good First Issue, Recommender System, PFIRec

BAB I

PENDAHULUAN

1.1. Latar Belakang

Open-source software (OSS) adalah paradigma yang menekankan sifat keterbukaan kode (*source code*), kolaborasi komunitas secara global dan partisipasi sukarela (*volunteering*) dalam pengembangan perangkat lunak melalui kanal seperti *repository*. Dalam praktiknya, proyek OSS telah mendorong inovasi dalam ekosistem perangkat lunak, hingga banyak produk tingkat industri yang memiliki ketergantungan pada proyek-proyek OSS seperti *repository* kernel Linux, ataupun sistem *database* PostgreSQL. Hal ini mengindikasikan implikasi holistik dari praktik pengembangan OSS pada stabilitas industri perangkat lunak.

Implikasi dari sifat kolaboratif proyek OSS adalah ketergantungan pada efektivitas dan jumlah kontributornya, dimana hal ini menjadi tantangan tersendiri jika proyek tidak dilengkapi dengan struktur organisasi yang kuat. Salah satu tantangan utama yang kerap muncul adalah tingginya tingkat pergantian kontributor, yang cenderung meningkat seiring dengan ukuran dan popularitas proyek (Zhou and Mockus, 2012). Faktor penyebabnya antara lain minimnya dukungan dari pengembang inti serta ketidaksesuaian antara kemampuan atau minat pengembang dengan isu yang tersedia. Sementara itu, tim inti pengembang (*core contributor*) yang umumnya berjumlah kecil juga mengalami kesulitan dalam merekrut anggota baru dan memberikan pembinaan yang efektif. Model *technical risk* proyek OSS (Schueller and Wachs, 2024) mengindikasikan kebutuhan proyek OSS akan jumlah kontributor yang stabil karena kehilangan satu hingga dua *core contributor* dapat mengakibatkan ketidakstabilan proyek seperti yang dialami proyek *web server* Spinner/Roxen.

Beberapa strategi untuk mengurangi friksi pada proses *onboarding* proyek OSS seperti dokumen README dan *Contributing Guidelines* telah terbukti efektif karena dapat memberi konteks proyek secara singkat pada calon kontributor. Adapun, strategi yang bersifat eksplisit dalam mengatasi permasalahan suatu proyek adalah label *Good First Issue* (GFI) yang biasanya diterapkan secara manual oleh *core contributor* melalui platform seperti Github

pada *issue* dengan kompleksitas rendah, dokumentasi yang jelas, serta dampak yang signifikan terhadap proyek (Zhou and Mockus, 2012).

Meski demikian, prosedur pelabelan GFI saat ini masih memiliki banyak keterbatasan. Label sering kali diberikan secara manual dengan preferensi subjektif yang tidak mempertimbangkan latar belakang teknis dari setiap individu, sehingga berpotensi menciptakan ketidaksesuaian teknis maupun kognitif antara *core maintainer* dan kontributor baru (Tan et al., 2020). Selain itu, pemberian label GFI di proyek OSS skala besar dapat menambah beban kurasi pada kelompok *core maintainer*.

Pendekatan berbasis kecerdasan artifisial dan pembelajaran mesin dapat digunakan untuk membantu pelabelan GFI dan pengalokasiannya ke kontributor baru. Secara hipotesis, kebutuhan sistem GFI akan mengotomatisasi proses dan analisis latar belakang kontributor sebagai dasar personalisasi *issue* dapat diselesaikan melalui pendekatan *artificial intelligence* dan *machine learning* (AI/ML) karena kemampuannya dalam analisis relasi data dan melakukan inferensi yang kompleks.

Pendekatan AI/ML dalam konteks rekomendasi *issue* telah diteliti sebelumnya. Secara evolusioner, studi terkait yang paling awal mengusulkan *Github Project Recommender* (Zebec, 2016) sebagai pendekatan *machine learning* berbasis *graph* dengan relasi kontributor dan proyek OSS sebagai *edge*. Meski belum spesifik mengarah pada rekomendasi *issue* yang sifatnya lebih atomik dalam proyek OSS, relevansi studi ini terdapat pada pendekatannya yang mempertimbangkan relasi dan latar belakang kontributor. Selanjutnya, terdapat studi yang beroperasi lebih spesifik pada tingkat *issue recommender* dengan mengusulkan model berbasis BERT (*Bidirectional Encoder Representations from Transformers*): GiveMeLabeledIssue (Vargovich et al., 2023). Model ini berfokus pada klasifikasi *issue* berdasarkan kemampuan yang diperlukan melalui analisis artefak kode terkait. Studi terkait juga mengusulkan desain arsitektur sistem *end-to-end* meski model dasar yang diakomodasi belum secara langsung merekomendasikan *issue* bagi calon kontributor.

Salah satu pendekatan teoritis yang komprehensif dan menjadi fokus dalam tugas akhir ini adalah model *Personalized First Issue Recommender* (PFIRec) (Xiao et al., 2023) berbasis algoritma *ranking* LambdaMART yang mampu mencapai akurasi tinggi dalam konteks relevansi *issue*. Hal ini dapat dicapai karena pendekatan model PFIRec yang mempertimbangkan ratusan fitur dari berbagai dimensi seperti informasi *issue* melalui *metadata*, profil kontributor dan *core contributor* pembuat *issue* hingga riwayat Git dari berbagai kontributor yang terlibat dalam proyek OSS terkait.

Meskipun model PFIRec memiliki akurasi yang tinggi, implementasinya di lingkungan industri masih minim. Hal ini dikarenakan kebutuhan dataset yang besar dan terstruktur, sehingga perlu didukung oleh arsitektur sistem yang efisien dan infrastruktur yang memadai. Selain itu, minimnya adopsi model PFIRec di lingkungan produksi mengindikasikan tingkat *user acceptance* yang masih perlu diuji. Untuk menjembatani kesenjangan sifat teoritis model PFIRec dengan proses pengembangan perangkat lunak, diperlukan implementasi sistem *end-to-end* dalam bentuk aplikasi yang terstruktur dan efisien agar dapat mendukung konstruksi dataset dan proses inferensi model. Selain itu, untuk mengoptimalkan adopsi model PFIRec dan keberlanjutan sistem, diperlukan pendekatan aplikasi dengan tampilan ramah pengguna melalui pendekatan *user-centric design*.

Secara umum, topik riset ini memiliki relevansi kuat dengan *roadmap* penelitian di bidang Rekayasa Perangkat Lunak dan Data (RPLD) di program studi Ilmu Komputer FMIPA UGM. Relevansi tersebut dicapai dengan menggabungkan pendekatan teknik pengembangan perangkat lunak, *machine learning*, dan desain UI/UX. Riset ini diharapkan dapat memberikan kontribusi praktis dan akademik terhadap pengembangan sistem kontribusi OSS yang lebih inklusif dan berkelanjutan.

1.2. Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, terlihat bahwa salah satu tantangan krusial dalam ekosistem Open-source Software (OSS) adalah kesulitan dalam mempertahankan kontributor baru dan efektivitas *onboarding*. Meskipun

strategi seperti *Good First Issue* (GFI) telah diterapkan, terdapat keterbatasan dalam personalisasi dan otomatisasi, menyebabkan seringnya terjadi ketidaksesuaian antara issue yang tersedia dengan latar belakang kontributor, yang pada akhirnya menghambat retensi.

Selain itu, implementasi model rekomendasi *issue* berbasis AI/ML seperti PFIRec, meskipun menjanjikan akurasi tinggi secara teoritis, masih menghadapi kendala signifikan dalam penerapannya di lingkungan industri. Tantangan ini meliputi kompleksitas pengumpulan dan pengelolaan dataset, kebutuhan infrastruktur yang memadai, serta aspek *user acceptance* yang belum teruji. Berdasar dari permasalahan-permasalahan tersebut, dapat dirumuskan poin-poin masalah penelitian sebagai berikut:

1. Proyek *Open Source Software* (OSS) kesulitan dalam mempertahankan kontributor baru, akibat gagalnya onboarding melalui ketidaksesuaian issue dengan latar belakang kontributor terkait.
2. Strategi onboarding klasik maupun modern seperti proses labeling *Good First Issue* (GFI) dan rekomendasi GFI belum diterapkan dalam lingkungan industri perangkat lunak karena kompleksitas dataset dan kebutuhan infrastruktur yang memadai.
3. Implementasi *sistem end-to-end* yang mengadopsi pendekatan AI/ML seperti model PFIRec sebagai GFI Recommender memiliki tantangan pengembangan perangkat lunak dalam konteks tampilan yang user-centric, skalabilitas, hingga biaya keberlanjutan sistem.

1.3. Batasan Masalah

Untuk memastikan fokus dan keberhasilan penelitian ini dalam mengatasi rumusan masalah yang telah dipaparkan, penting untuk menetapkan batasan-batasan penelitian yang jelas. Pengembangan sistem *end-to-end* untuk model PFIRec, yang merupakan inti dari riset ini, memiliki cakupan yang luas. Oleh karena itu, batasan ini akan mengarahkan sumber daya dan upaya penelitian agar tetap terfokus pada aspek-aspek paling krusial.

Secara umum, pengembangan sistem *end-to-end* untuk model PFIRec dalam konteks penelitian ini melibatkan beberapa aspek sebagai berikut, dengan batasan yang telah ditentukan:

1. Analisis, *training* dan pengembangan API untuk model PFIRec.
2. Perancangan desain antarmuka pengguna (UI/UX Design) berbasis pendekatan *user-centered design* secara terbatas, dengan fokus pada efisiensi fungsional dan kebutuhan pengguna utama yang telah diasumsikan familiar dengan platform VCS seperti Github.
3. Pengembangan aplikasi *client-side*, *server-side* dan infrastruktur pendukung.
4. Pengujian sistem secara teknis meliputi *unit testing* (pengujian unit) dan *end-to-end testing* (pengujian menyeluruh).
5. Pengujian pengguna melalui *user-acceptance test* untuk mengukur efektivitas dari pendekatan yang diajukan dan potensi perbaikan di masa mendatang.

1.4. Tujuan Penelitian

Berdasarkan rumusan masalah dan batasan yang telah diidentifikasi, penelitian ini memiliki beberapa tujuan utama yang ingin dicapai. Tujuan-tujuan ini dirancang untuk secara langsung mengatasi tantangan yang ada dalam meningkatkan pengalaman *onboarding* kontributor baru di proyek OSS serta menjembatani kesenjangan antara teori AI/ML dengan implementasi praktis di industri. Secara spesifik tujuan dari penelitian dapat dijabarkan sebagai berikut.

1. Merancang arsitektur perangkat lunak untuk mendukung penggunaan model PFIRec dalam lingkungan industri.
2. Merancang desain UI/UX yang bersifat intuitif, *user-centric* dan berfokus pada alur kontribusi kontributor baru.
3. Mengevaluasi tingkat adopsi sistem di lingkungan industri melalui pengujian performa serta *user acceptance rate* di berbagai demografi pengembang perangkat lunak.

1.5. Manfaat Penelitian

Implementasi dan integrasi sistem *end-to-end* untuk model PFIRec diharapkan dapat memberikan manfaat yang signifikan dan berdampak luas bagi berbagai pihak. Manfaat ini tidak hanya terbatas pada komunitas *open-source*, tetapi juga akan memberikan kontribusi penting bagi praktisi akademik dan pengembangan mendatang. Secara khusus, manfaat dari penelitian ini meliputi:

1. Penyediaan platform yang lebih adaptif dan personal bagi komunitas *open-source contributor* yang akan meningkatkan pengalaman *onboarding* kontributor baru, sehingga dapat meningkatkan retensi kontributor.
2. Peningkatan *output* proyek OSS dan stabilitas industri perangkat lunak secara keseluruhan yang diperoleh dari peningkatan retensi kontributor.
3. Menjadi bentuk pendekatan konkret dalam menjembatani model AI/ML yang bersifat teoritis dengan lingkungan industri yang memiliki tantangan teknis maupun tingkat adopsi berdasarkan pengalaman pengguna.
4. Menjadi dasar penelitian lebih lanjut dalam pengembangan lingkungan *open-source software* (OSS) yang lebih inklusif dan berkesinambungan.

BAB II

TINJAUAN PUSTAKA

2.1. Git

Dalam pengembangan perangkat lunak modern, kolaborasi dan manajemen perubahan kode merupakan aspek krusial. Git adalah sistem kontrol versi terdistribusi (*Distributed Version Control System/DVCS*) yang dirancang untuk menangani proyek dengan ukuran apa pun, dari yang kecil hingga sangat besar, dengan kecepatan dan efisiensi. Diciptakan oleh Linus Torvalds pada tahun 2005 untuk pengembangan kernel Linux, Git telah menjadi standar *de facto* dalam dunia pengembangan perangkat lunak, terutama dalam proyek *open-source*.

Berbeda dengan sistem kontrol versi terpusat (seperti SVN atau CVS), Git memungkinkan setiap pengembang memiliki salinan *repository* lengkap (*full repository*) di perangkat lokal mereka. Hal ini berarti setiap operasi dapat dilakukan secara lokal tanpa perlu koneksi ke *server* pusat, memberikan fleksibilitas dan kecepatan yang lebih baik. Konsep inti Git meliputi:

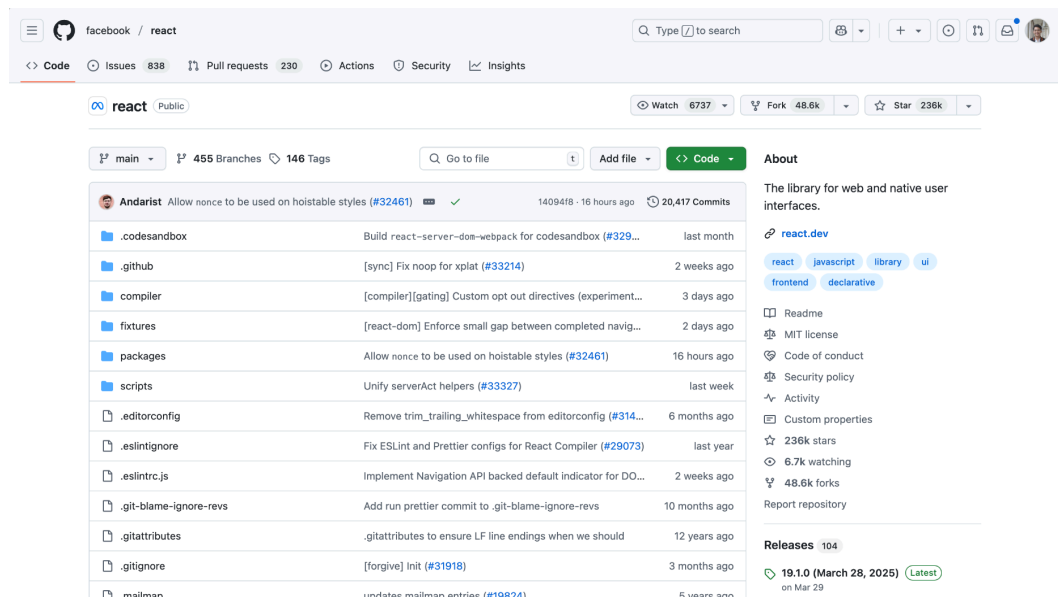
- **Repository (Repo)** yang merupakan direktori kerja proyek beserta semua *file* yang terkait, riwayat revisi lengkap, dan metadata khusus Git. Setiap *repository* lokal adalah salinan lengkap dari *repository* pusat (*remote*), memungkinkan operasi lokal dan kolaborasi terdistribusi.
- **Commit** yang merupakan *snapshot* dari perubahan kode pada waktu tertentu yang disimpan dalam riwayat *repository*. Setiap *commit* memiliki pengenal unik (hash), pesan deskriptif, dan mereferensikan *commit* sebelumnya, membentuk rantai riwayat perubahan.
- **Branch** adalah penunjuk ke *commit* tertentu yang memungkinkan pengembang untuk membuat jalur pengembangan terpisah dari *branch utama*. *Branch* umumnya digunakan untuk mengerjakan fitur baru atau perbaikan *bug* secara paralel tanpa memengaruhi kode utama yang stabil.
- **Merge** adalah penggabungan perubahan dari satu *branch* ke *branch* lain. Operasi ini menciptakan sebuah *commit* baru yang menyatukan riwayat dari kedua *branch* yang digabungkan.

- **Pull** merupakan operasi pengambilan (*fetch*) perubahan terbaru dari *repository remote* dan menggabungkannya (*merge*) ke *repository* lokal.
- **Push** merupakan kebalikan dari operasi *pull* dan digunakan untuk mengirimkan (*upload*) perubahan yang telah di-*commit* dari *repository* lokal ke *repository remote*.

Fleksibilitas dan sifat terdistribusi Git memungkinkan alur kerja kolaboratif yang kuat dan tangguh, menjadikannya fondasi bagi platform pengembangan kolaboratif seperti GitHub.

2.2. GitHub

GitHub adalah platform *hosting repository* Git berbasis *web* yang sangat populer, menyediakan layanan *version control*, kolaborasi, dan manajemen proyek. Diluncurkan pada tahun 2008, GitHub telah menjadi pusat bagi jutaan proyek *open-source* dan pengembangan perangkat lunak privat, memungkinkan pengembang dari seluruh dunia untuk berkolaborasi secara efisien. Tampilan umum dari *platform* Github (halaman suatu *repository*) dapat divisualisasikan dengan Gambar 2.1.



Gambar 2.1. Contoh Tampilan *Repository* react di *platform* web GitHub

Selain digunakan sebagai penyimpanan berkas kode, GitHub juga memiliki serangkaian fitur yang mendukung siklus pengembangan perangkat

lunak, terutama dalam konteks kolaborasi. Alur kerja kontribusi utama di GitHub umumnya mengikuti langkah-langkah berikut:

1. **Identifikasi *Issue***

Proses kontribusi sering dimulai dengan mengidentifikasi sebuah **issue** yang relevan dalam proyek. *Issue* ini bisa berupa laporan *bug*, permintaan fitur baru, pertanyaan, atau diskusi umum terkait pengembangan. Pengembang dapat menelusuri daftar *issue* yang ada untuk menemukan tugas yang sesuai dengan keahlian atau minat mereka.

2. **Kloning *Repository* dan Pengembangan Lokal**

Setelah *issue* dipilih, kontributor akan mengkloning *repository* proyek ke perangkat lokal mereka menggunakan Git. Kemudian, kontributor akan membuat *branch* baru untuk mengerjakan perubahan terkait *issue* tersebut. Semua pengembangan, modifikasi kode, dan penambahan fitur dilakukan di *branch* lokal ini.

3. **Melakukan *Commit***

Setiap kali kontributor menyelesaikan sebagian kecil dari pekerjaan atau mencapai titik logis dalam pengembangan, mereka akan menyimpan perubahannya sebagai **commit** di *branch* lokal mereka. Setiap *commit* disertakan dengan pesan yang jelas menjelaskan perubahan yang dibuat.

4. **Mengajukan *Pull Request (PR)***

Setelah pekerjaan pada *branch* lokal selesai dan siap untuk digabungkan ke *repository* utama, kontributor akan mengajukan *Pull Request (PR)* melalui *interface* GitHub. PR ini adalah proposal untuk menggabungkan *branch* kontributor ke *branch* target (biasanya *main branch*). PR berfungsi sebagai wadah diskusi, tinjauan kode, dan serangkaian pengujian otomatis.

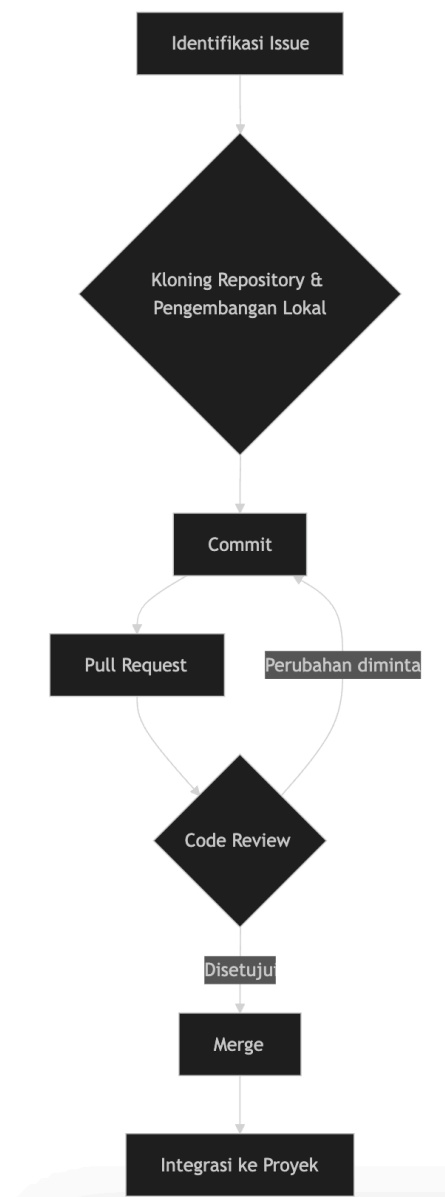
5. **Proses *Code Review***

Setelah PR diajukan, *core maintainer* atau kontributor lain yang ditunjuk akan melakukan *code review*. Mereka akan meninjau perubahan kode yang diusulkan, memberikan umpan balik, meminta perbaikan, atau menyetujui perubahan. Tahap ini krusial untuk memastikan kualitas, konsistensi, dan kepatuhan kode terhadap standar proyek.

6. Penggabungan (*Merge*)

Jika *pull request* telah disetujui setelah melalui proses *code review* dan semua *check* otomatis (misalnya *Continuous Integration*) telah berhasil, *core maintainer* akan melakukan *merge*.

Secara umum, proses-proses kolaborasi di GitHub berimplikasi pada terciptanya sebuah alur kerja yang terstruktur dan transparan sehingga dapat memfasilitasi kerja tim serta memastikan kualitas kode yang terjaga. Visualisasi dari operasi-operasi kolaboratif di GitHub dapat didefinisikan dalam Gambar 2.2.



Gambar 2.2. Diagram Proses Operasi Kolaboratif di GitHub

2.3. *Good First Issue (GFI)*

Good First Issue (GFI) merupakan label yang diberikan umumnya oleh *core contributor* dalam proyek OSS untuk menunjukkan *issue* yang dinilai cocok bagi kontributor baru. Label ini biasanya diterapkan pada *issue* dengan kompleksitas rendah, dokumentasi yang jelas, serta dampak yang signifikan terhadap proyek, sehingga memungkinkan kontribusi yang cepat dan efektif (Zhou and Mockus, 2012). Penugasan eksplisit melalui *issue* dinilai dapat meningkatkan kemungkinan kontributor untuk menyelesaikan task dibandingkan hanya dengan mengandalkan dokumentasi umum seperti README atau *contributing guideline* (Steinmacher et al., 2014)

Implementasi GFI dalam *platform Version Control System (VCS)* seperti Github umumnya dimulai dari *core maintainer* yang merilis *issue* dan sebaiknya dilengkapi dengan label *Good First Issue*, label pendukung lain seperti jenis teknologi atau proses bisnis hingga deskripsi yang komprehensif. Kontributor yang ingin bergabung ke dalam proyek terkait, akan memberi komentar konfirmasi di *issue* terkait, melakukan pengembangan sesuai deskripsi, lalu menghubungkannya dengan *pull request*. Dari skema tersebut, terlihat bahwa sistem GFI bersifat praktis karena langsung terhubung dengan proses pengembangan perangkat lunak di *platform VCS*.

Namun, pendekatan GFI juga memiliki sejumlah keterbatasan. Pertama, proses pemberian label GFI umumnya dilakukan secara manual oleh *core maintainer*, sehingga sangat bergantung pada subjektivitas dan konsistensi masing-masing individu. Hal ini dapat menimbulkan masalah ketidakseragaman dalam kualitas pembuatan *issue* dan kekurangan akomodasi waktu dari *core maintainer*. Kedua, label GFI yang tidak mempertimbangkan latar belakang atau preferensi individual kontributor dapat menimbulkan ketidaksesuaian antara kemampuan kontributor dengan kompleksitas atau konteks isu yang disarankan (Tan et al., 2020). Terakhir, isu-isu yang telah lama diberi label GFI dapat menjadi usang (*stale*) apabila tidak diperbarui secara berkala, baik karena *dependency* yang telah berubah maupun konteks yang tidak lagi relevan.

2.4. Issue Recommender Berbasis AI/ML

2.4.1. Github Project Recommender

Upaya memfasilitasi koneksi antara kontributor dengan proyek OSS yang relevan merupakan tantangan signifikan di platform kolaboratif besar seperti GitHub karena volume aktivitasnya yang sangat besar. Oleh karena itu, sistem rekomendasi proyek yang efektif penting dalam meningkatkan keterlibatan kontributor. Menjawab kebutuhan ini, diusulkan model *Github Project Recommender* (Zebec, 2016) secara khusus menyelidiki detail implementasi teknik analisis *graph* untuk pengembangan sistem rekomendasi proyek OSS di platform GitHub. Pendekatan tersebut berfokus pada pemodelan relasi pengguna dan proyek sebagai sebuah *bipartite graph* dan implementasi teknik (*link prediction* untuk mengidentifikasi koneksi yang berpotensi).

Dalam proses analisis, pendekatan tersebut mengusulkan penyusunan *bipartite graph* yang merepresentasikan interaksi antara pengguna GitHub dan *repository*. Studi terkait menggunakan 1.000 *repository* teratas dalam konteks *ranking* jumlah kontributor unik yang melakukan *commit*, dan akumulasi kontributor yang telah berkontribusi pada *repository-repository* melalui *commit* atau *pull request*. Dalam proses analisis, studi yang diusulkan berkonsentrasi pada komponen *node* terhubung yang terbesar (*largest connected component*) yang terdiri dari sekitar 72.088 *node* (kontributor dan *repository*) dan 91.385 *link*.

Setelah itu, dilakukan evaluasi sistematis beberapa metode *link prediction* yang diadaptasi untuk struktur bipartit menggunakan rumus *Area Under the ROC Curve* (AUC) untuk mengukur kinerja. Indeks *similarity* lokal (contohnya *Common Neighbors*, *Jaccard Index*, *Adamic-Adar*, dll.), diterapkan dengan definisi *neighborhood* yang dimodifikasi untuk *node* kontributor (mempertimbangkan *node* yang berjarak dua langkah), secara umum hanya menghasilkan akurasi yang cukup baik (nilai AUC sekitar 0.60). Meski pendekatan dengan *fine tuning* pada *weights* dari indeks *Preferential Attachment* dan *Adamic-Adar* menunjukkan hasil akurasi yang lebih baik (AUC = 0.89), pendekatan yang terbukti paling konsisten dan efektif adalah pendekatan yang mempertimbangkan struktur komunitas di Github. Dengan mengimplementasikan

algoritma Infomap untuk mendeteksi komponen *node* yang membentuk “komunitas” dalam graf, nilai mencapai AUC tertinggi sebesar 0.90.

Studi ini menyimpulkan bahwa metode *link prediction* yang berbasis komponen komunitas dalam representasi *bipartite graph* memiliki potensi signifikan dalam merekomendasikan proyek GitHub. Pendekatan yang diusulkan secara efektif menangkap kecenderungan kontributor yang terlibat dalam satu proyek untuk berkolaborasi atau tertarik pada proyek lain dalam kluster topikal atau sosial yang sama. Namun, keterbatasan studi ini terletak pada hasil akurasi yang tinggi tidak mampu digeneralisasi dengan baik di luar *dataset* khusus (potensi *overfitting*). Selain itu, keterbatasan model ini terletak pada penggunaan riwayat relasi kontributor dengan proyek sebagai *edge* dalam *graph* sehingga secara implisit mengharuskan pengguna untuk memiliki pengalaman pada proyek yang relevan.

2.4.2. GiveMeLabeledIssues

GiveMeLabeledIssues (Vargovich et al., 2023) merupakan sistem *issue recommender* dalam proyek OSS yang dirancang untuk menanggapi tantangan umum yang dihadapi kontributor dalam menavigasi sistem *issue tracker* di dalam proyek *Open Source Software* (OSS) guna menemukan task yang sesuai dengan keterampilan dan preferensi teknis mereka. Berbeda dengan proses *labeling issue* tradisional yang berfokus pada tipe *issue*, studi ini mengusulkan pendekatan baru dengan memberi label *issue* berdasarkan keterampilan teknis yang dibutuhkan untuk penyelesaiannya. Hal ini dicapai melalui analisis domain dari API yang terlibat dalam solusi terkait yang pernah diajukan (melalui *pull request*) sebagai media bagi jenis keahlian yang diperlukan (misalnya UI, *Database*, atau *Testing*). Dengan menyediakan label berbasis keterampilan, model dapat memfasilitasi pencocokan kapabilitas kontributor dengan *task* yang cocok, sehingga dapat mengurangi beban bagi *core contributor* proyek dan meningkatkan pengalaman *onboarding* yang sederhana dan intuitif bagi kontributor baru. Selain itu, studi ini berusaha menyediakan solusi *end-to-end* yang praktis untuk penemuan *issue* yang disesuaikan dengan kemampuan pengembang individual.

Pendekatan yang diusulkan merupakan implementasi arsitektur *end-to-end* yang dirancang untuk penerapan praktis. Sistem GiveMeLabeledIssues terstruktur dalam dua lapisan utama: antarmuka *web frontend* yang dibangun dengan *framework* Angular dan *backend* REST API. Bagian *back-end* yang bertanggung jawab untuk pemrosesan data yang berat, termasuk proses *data mining* dari *repository* proyek OSS untuk data *issue* dan *pull request* (PR), mengurai *source code* untuk mengidentifikasi API, membangun *training dataset*, dan memanggil model *machine learning* TF-IDF dengan variasi pendekatan dengan *Random Forest* (scikit-learn) maupun BERT (fast-bert).

Bagian penting dari pendekatan *backend* di sistem yang diusulkan adalah proses harian yang secara aktif mengklasifikasikan *issue* yang saat ini terbuka dan menyimpan hasilnya *database* SQLite. Antarmuka *front-end* dalam sistem mengonsumsi data analisis melalui protokol REST API, sehingga memungkinkan pengguna untuk memilih proyek, menentukan keterampilan mereka, dan menerima daftar *issue* terbuka yang relevan dan telah di-*filter*.

Meski mengusulkan implementasi *end-to-end* yang komprehensif, studi yang diusulkan masih memiliki beberapa keterbatasan. Keterbatasan yang utama adalah desain sistem *end-to-end* yang memerlukan model terisolasi untuk setiap proyek OSS yang terdaftar. Hal ini menimbulkan risiko inefisiensi dari kebutuhan dataset spesifik tiap proyek dan sumber daya *training* model yang spesifik. Selain itu, proses reklasifikasi harian yang dijalankan komponen *back-end* pada *issue-issue* terbuka akan menambahkan biaya tinggi dari komputasi yang berulang. Keterbatasan yang terakhir adalah kemungkinan penurunan akurasi model dari kekurangan data akibat proses pengembangan di platform Github yang kerap menyelesaikan *issue* tanpa melalui proses *pull request* formal.

2.4.3. PFIRec

Personalized First Issue Recommender (PFIRec) (Xiao et al., 2023) merupakan sebuah model *recommender* untuk menyediakan rekomendasi *issue* pertama yang dipersonalisasi bagi kontributor baru, dengan asumsi bahwa

rekomendasi yang dipersonalisasi dapat secara signifikan meningkatkan efektivitas dan mengurangi upaya yang tidak berhasil.

Studi yang diusulkan menganalisis data untuk mendukung asumsi personalisasi dengan mengidentifikasi faktor-faktor yang berpengaruh. Studi melakukan analisis empiris terhadap 37 kontributor baru dan 123 *first issue* yang telah mereka selesaikan di berbagai proyek OSS. Hasil dari studi ini memberikan bukti empiris bahwa pilihan *issue* pertama kontributor baru cenderung bersifat personal, menunjukkan konsistensi dalam tipe *task*, bahasa pemrograman, dan domain proyek. Selanjutnya, penelitian tersebut mengidentifikasi bahwa preferensi kemampuan (terkait dengan riwayat kontribusi), pengalaman kontribusi dalam komunitas OSS, keaktifan dalam komunitas OSS (*activeness*), hingga sentimen kontributor baru merupakan fitur-fitur yang mempengaruhi pilihan *issue* pertama mereka yang dipersonalisasi.

Berdasarkan data empiris tersebut, studi tersebut mengusulkan model peringkat (*ranking model*) PFIRec yang mengimplementasikan algoritma LambdaMART. LambdaMART adalah algoritma *learning-to-rank* yang membangun *decision tree ensemble* untuk mengoptimalkan metrik peringkat secara langsung, seperti *Normalized Discounted Cumulative Gain* (NDCG). Dalam PFIRec, LambdaMART digunakan untuk memberi peringkat pada daftar kandidat *issue* berdasarkan relevansinya bagi kontributor baru. Model ini menggabungkan serangkaian fitur yang komprehensif, termasuk fitur-fitur yang diekstraksi dari profil dan aktivitas kontributor baru (preferensi keahlian, pengalaman OSS umum, keaktifan, sentimen) serta fitur-fitur dari *issue* kandidat itu sendiri (analisis tekstual dari judul dan badan *issue* menggunakan representasi seperti TF-IDF atau *embeddings*, metadata *issue* seperti label, jumlah komentar, tanggal dibuat, serta profil pelapor *issue*). Fungsi *ranking* yang dipelajari oleh LambdaMART dapat menghasilkan skor relevansi untuk setiap pasangan kontributor dan *issue*, yang pada akhirnya digunakan untuk mengurutkan *issue* kandidat.

Evaluasi model PFIRec menggunakan *dataset* yang terdiri dari 68.858 *issue* dari 100 proyek GitHub menunjukkan bahwa PFIRec secara signifikan

mengungguli pendekatan *issue recommender* berbasis GFI sebelumnya dalam hal akurasi peringkat dan potensi peningkatan tingkat penyelesaian *issue* pertama. Namun, studi yang diusulkan juga menyebutkan beberapa keterbatasan. Efektivitas model sangat bergantung pada ketersediaan dan frekuensi data historis kontributor baru untuk melatih model personalisasi yang akurat. Adapun, sumber data dari fitur-fitur yang relevan memerlukan proses *data mining* dan *data processing* yang ekstensif dari platform GitHub. Selain itu, model ini pada dasarnya adalah model peringkat yang memerlukan serangkaian *issue* kandidat untuk dievaluasi, yang efisiensinya dapat bervariasi tergantung pada jumlah *issue* terbuka dalam sebuah proyek.

2.5. Github API

Github *Application Programming Interface* (API) adalah layanan resmi dari GitHub bagi aplikasi pihak ketiga untuk mengakses dan memanipulasi data di platform Github. Layanan terkait menjadi komponen fundamental dalam pengembangan sistem yang terhubung dengan data GitHub, data *repository*, *issue*, riwayat kontribusi, hingga *metadata*. Secara umum, Github menyediakan tiga jenis API yang dapat diakses publik dengan otorisasi PAT (*Personal Access Token*) dengan batasan akses sebesar 5.000 *request* per jam yaitu Github REST API, Github GraphQL API, dan yang terbaru, Github MCP Server.

Github REST API, dengan versi terkini v3, mengikuti arsitektur RESTful dan paling luas digunakan di lingkungan perangkat lunak karena kompatibilitas sistem dan familiaritas dengan pengembang yang tinggi. Github REST API menggunakan format permintaan HTTP standar (GET, POST, PATCH, DELETE) dan merespons dalam format JSON sederhana. Meski demikian, Github REST API memiliki keterbatasan yang diturunkan dari arsitektur RESTful yaitu sulit untuk mendapatkan data dengan relasi maupun risiko inefisiensi akibat *over/under fetching* dimana jumlah data yang dikembalikan *server* tidak sesuai dengan yang dibutuhkan.

Jenis API selanjutnya adalah Github GraphQL API yang berbasis pada arsitektur GraphQL sehingga mampu menyelesaikan keterbatasan Github REST

API dengan fitur kueri dimana pengguna dapat meminta jenis dan jumlah data dengan akurat pada *server*. Meski demikian, arsitektur GraphQL yang belum banyak didukung secara otomatis pada mayoritas sistem mengakibatkan adopsi Github GraphQL tidak terlalu luas (Jin et al., 2024).

Jenis API yang paling baru adalah Github MCP (*Machine Context Protocol*) Server yang dikembangkan sebagai jalur integrasi aplikasi LLM (*Large Language Model*) dengan *server* Github. Kelebihan MCP Server adalah kemampuannya menerima input berupa *natural language*, menerjemahkannya menjadi struktur API formal dan mengembalikan data yang relevan, sehingga cocok untuk diintegrasikan dengan LLM sebagai konteks dan sumber data. Akan tetapi, latensi dari Github MCP Server lebih lambat secara objektif karena pemrosesan *natural language* yang perlu terjadi di dalam *server*.

2.6. Database

Database atau basis data dapat didefinisikan secara formal sebagai kumpulan data yang terorganisir dan saling terkait melalui relasi yang memodelkan topik atau *domain* tertentu. Konsep *database* berbeda dan diperluas dengan *Database Management System* (DBMS), yang merupakan perangkat lunak yang digunakan untuk membuat, memelihara, dan mengelola *database*. Tujuan utama dari penggunaan DBMS adalah mengatasi tantangan dalam manajemen data seperti redundansi, inkonsistensi, kesulitan akses, integritas data, dan masalah konkurensi (Silberschatz et al., 2011).

Salah satu pendekatan *database* tradisional adalah *database* relasional (SQL) yang dikonseptualisasikan sebagai susunan data logis di dalam tabel (relasi) yang terdiri dari baris (tupel) dan kolom (atribut), dengan relasi yang secara eksplisit didefinisikan menggunakan *keys*. Karakteristik dari *database* relasional adalah susunan data terstruktur melalui *schema* yang telah ditentukan sebelumnya. Selain itu, *database* SQL mengimplementasikan mekanisme untuk memastikan integritas data yang kuat (contohnya *primary* dan *foreign* key, *constraints*, hingga *normalization*). Dalam hal akses data, *database* relasional memiliki jaminan *transaction* kuat yang dideskripsikan melalui konsep sifat

ACID (*Atomicity, Consistency, Isolation, Durability*) (Haerder dan Reuter, 1983). Namun, *database* relasional memiliki keterbatasan dalam menanggapi beberapa kebutuhan khusus dalam lingkungan komputasi modern seperti adanya kompleksitas tinggi untuk mencapai skalabilitas secara horizontal yang hemat biaya dan desain *schema* yang tidak fleksibel terhadap perubahan *requirement* proyek atau data.

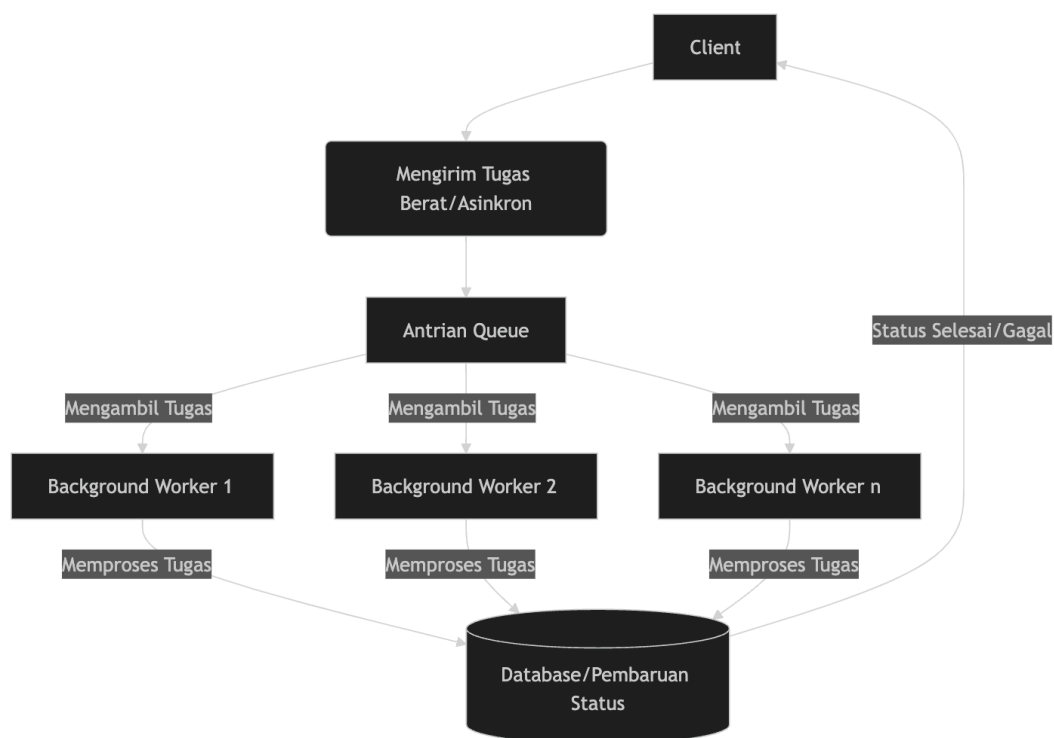
Menanggapi keterbatasan dari *database* relasional, terutama dalam ranah *Big Data* dan aplikasi dengan skalabilitas tinggi, muncul pendekatan NoSQL yang bervariasi (Gomes et al., 2023). Karakteristik utama dari *database* NoSQL adalah fleksibilitas skema yang memungkinkan struktur data yang bervariasi, arsitektur yang dirancang untuk skalabilitas horizontal dan sistem terdistribusi, dan umumnya memprioritaskan ketersediaan (*availability*) dan performa yang tinggi dibandingkan konsistensi dan integritas data. Untuk mencapai karakteristik tersebut, sistem NoSQL umumnya melonggarkan sifat SQL yaitu jaminan ACID yang ketat menjadi konsep BASE (*Basically Available, Soft State, Eventually Consistent*). Adapun, turunan dari pendekatan MySQL dimanifestasikan dalam variasi seperti model *key-value* (Redis), *document* (Firestore, dan MongoDB), *vector*, *graph* dan lainnya. Meski demikian, implementasi sistem NoSQL memiliki tantangan tersendiri seperti kompleksitas tambahan untuk mencapai konsistensi dan integritas data tinggi, sehingga kerap kali dibebankan pada *application layer* dalam implementasi mekanisme integritas data (Asaad et al., 2020).

2.7. Pola *Queue* dan *Background Worker*

Queue dan *background worker* adalah kombinasi pola arsitektur dalam sistem *back-end* yang digunakan untuk menangani proses berat atau berjangka panjang tanpa mengganggu alur utama aplikasi yang bersifat interaktif ataupun *real-time*. Dasar dari pola ini adalah pemisahan beban kerja proses yang penting namun berukuran besar, lambat, atau terjadwal dari proses utama (Qi Zhang et al., 2006). Beberapa contoh penggunaan umum dari pola ini mencakup pengambilan data agregat dari layanan eksternal, pengolahan *file* besar, pengiriman email

massal, penjadwalan tugas secara periodik, hingga pemrosesan inferensi model AI/ML.

Secara konseptual, mekanisme pola ini akan menempatkan tugas-tugas ke dalam sebuah antrian (*queue*), yang kemudian diproses secara terpisah oleh sejumlah komponen *worker*. *Queue* berfungsi sebagai *buffer* antara proses utama dengan komputasi berat. Sedangkan *worker* akan secara paralel dan berkala mengambil tugas dari *queue*, mengeksekusinya, dan memperbarui status. Selain itu, kombinasi pola ini memungkinkan variasi arsitektur dengan kontrol dan pengelolaan proses yang komprehensif seperti pemantauan status (*monitoring*), *retry* otomatis ketika ada kegagalan proses individu hingga pengatur prioritas antrian proses (Tsitsiklis and Xu, 2017). Keseluruhan proses umum dari komponen *queue* dan *background worker* dapat divisualisasikan dalam Gambar 2.3. sebagai berikut.



Gambar 2.3. Diagram Proses dari Pola *Queue* dan *Background Worker*

Namun, penerapan pola *queue* dan *background worker* juga memiliki kompleksitas inheren yang tinggi karena mekanisme pengelolaan antrian,

pemrosesan paralel, sinkronisasi status, hingga penanganan potensi duplikasi atau kegagalan proses. Tanpa desain arsitektur yang mumpuni, sistem dapat mengalami penumpukan beban kerja, inkonsistensi data, atau bahkan *bottleneck* baru di beberapa lapisan. Untuk mengurangi kompleksitas tersebut, tersedia berbagai *framework* yang menggeneralisasi kompleksitas dari *queue* dan telah teruji di lingkungan industri, seperti Celery (berbasis Python), BullMQ (Node.js), dan Redis Queue (RQ).

2.8. Sistem *Front-End*

Komponen *front-end* atau *client-side* dalam konteks arsitektur sistem *end-to-end* dapat didefinisikan sebagai tampilan / *interface* visual yang menjadi mediator interaksi pengguna dan proses *server-side* di latar belakang. Selain konsep visual yang diturunkan dari proses desain UI/UX (*User Interface / User Experience*), komponen *front-end* berperan penting dalam pengaturan variasi masukan dari pengguna, menampilkan *feedback* dari proses *server-side* hingga menjaga keamanan sistem di lapisan konseptual paling luar.

Batasan teknis dari sistem *front-end* mencakup beberapa dimensi. Dimensi pertama adalah manajemen *state*, dimana *state* dapat diartikan sebagai kondisi aplikasi *front-end* relatif terhadap proses yang sedang berjalan di aplikasi *server-side*. Sebagai contoh, *state management* perlu dikembangkan agar pengguna dapat melihat *feedback* melalui tampilan visual yang informatif dan koheren dengan proses yang sedang berjalan (contohnya memuat data, ataupun penanganan galat). Dimensi selanjutnya adalah performa aplikasi yang mencakup optimasi tampilan (*rendering*) dan *caching* data yang sering diakses agar cepat dimuat. Dimensi yang terakhir adalah keamanan aplikasi dimana *client-side* sebagai lapisan pertama yang diakses pengguna perlu mengimplementasikan mekanisme keamanan seperti enkripsi data, mitigasi *cross-site scripting* (XSS) dan *content security policy* (CSP).

Implementasi sistem *front-end* modern yang kompleks mendorong pengembangan berbagai *tool*, *framework* (kerangka kerja) atau *library* guna menggeneralisasi suatu konsep solusi. Hal ini esensial karena optimasi teknologi

front-end secara umum dapat meningkatkan performa aplikasi dan *user experience* (Xiaoshu, 2020). Salah satu *framework* yang populer digunakan dalam lingkungan pengembangan web modern adalah NextJS yang berbasis *library* ReactJS dan bahasa pemrograman Javascript. NextJS sebagai *framework* berhasil meningkatkan kapabilitas ReactJS melalui fitur optimasi seperti *server-side rendering* (SSR) yang meningkatkan kecepatan memuat data dan visibilitas situs di *search engine* (Ekpobimi, 2024). Selain *framework* atau *library*, terdapat *tool* lain seperti TailwindCSS yang menawarkan pengembangan tampilan UI web secara sistematis dan lebih cepat.

2.9. User-Centered Design

User-Centered Design (UCD) merupakan metodologi desain yang bersifat sistematis dengan menekankan pemahaman mendalam dan keterlibatan aktif pengguna pada tahapan-tahapan relevan dalam proses pengembangan produk. Tujuan utama dari metodologi UCD adalah untuk memastikan kesesuaian sistem dengan kebutuhan interaksi, ekspektasi, dan keterbatasan pengguna (Chammas et al., 2015). Dalam konteks pengembangan aplikasi yang telah ditentukan, konsep UCD diimplementasikan secara dominan pada tahapan ideasi dan desain prototipe antarmuka pengguna (UI/UX). Pendekatan ini juga menjadi acuan dalam evaluasi terhadap desain antarmuka tersebut.

Dalam praktiknya, penerapan UCD untuk perancangan antarmuka dimulai dengan tahap penelitian pengguna yang terfokus. Teknik seperti wawancara, survei, dan pengembangan *user persona* dapat digunakan untuk membentuk profil pengguna, yang mencakup preferensi perilaku mereka dalam berinteraksi dengan sistem serupa, serta pemahaman terhadap alur kerja mereka yang relevan dengan fungsionalitas sistem yang akan dirancang. Berdasarkan pemahaman tersebut, tahap ideasi dilakukan untuk menghasilkan konsep-konsep desain antarmuka yang bertujuan untuk memudahkan pengguna dalam memanfaatkan fungsionalitas sistem yang ada secara efektif. Konsep-konsep ini kemudian diterjemahkan ke dalam bentuk desain prototipe melalui praktik desain UI/UX. UI (*User Interface*) mengacu pada aspek visual dan interaktif dari suatu sistem, sementara UX (*User*

Experience) mengkaji keseluruhan pengalaman interaksi pengguna terhadap produk tersebut. Dalam konteks UCD, praktik desain UI/UX dituntut untuk mencapai hasil desain yang menarik secara estetika, namun yang terpenting adalah mampu menanggapi kebutuhan fungsional spesifik dari sistem yang ada dan memastikan kenyamanan pengguna dalam berinteraksi dengannya.

Selama siklus desain dan pengembangan antarmuka, evaluasi dapat dilakukan secara berulang terhadap prototipe untuk memastikan bahwa setiap iterasi desain membawa peningkatan nyata terhadap pengalaman pengguna. Dari langkah-langkah proses UCD, dapat divisualisasikan diagram di Gambar 2.3. sebagai berikut.



Gambar 2.4. Diagram Proses Pendekatan *User-Centered Design*

Dengan demikian, UCD dalam konteks ini berperan sebagai kerangka kerja strategis yang mengarahkan proses perancangan dan evaluasi antarmuka pengguna untuk sebuah sistem dengan fungsionalitas inti yang telah ditetapkan.

BAB III

METODE DAN RANCANGAN PENELITIAN

3.1. Deskripsi Umum Penelitian

Penelitian ini berfokus pada pengembangan produk perangkat lunak *end-to-end* yang mengimplementasikan model *issue recommender* PFIRec dalam konteks penggunaan nyata di lingkungan industri perangkat lunak. Berbeda dengan metodologi penelitian yang menekankan performa algoritmik di lingkungan akademik, penelitian ini bertujuan untuk merealisasikan sistem yang dapat digunakan, dioperasikan, dan dievaluasi secara langsung oleh kontributor *open-source* pada platform berbasis Git seperti Github. Pendekatan ini sejalan dengan kebutuhan komunitas kontributor OSS yang memerlukan solusi konkrit untuk meningkatkan efektivitas *onboarding*, retensi, dan kontribusi berkelanjutan.

Model PFIRec dipilih sebagai dasar logika *recommender* karena model ini memiliki cakupan data yang komprehensif dengan melibatkan 120 fitur yang mencakup profil calon kontributor, karakteristik *issue*, serta profil pelapor *issue*. Namun, keunggulan model ini juga menimbulkan tantangan teknis berupa kebutuhan infrastruktur pendukung yang memadai mulai dari pengambilan dan pemrosesan data, stabilitas inferensi, hingga penyajian rekomendasi melalui antarmuka yang intuitif. Hal ini menyebabkan model PFIRec yang teruji memiliki performa cukup baik di lingkungan akademis sulit divalidasi tingkat penerimaannya di lingkungan produksi dengan pengguna riil.

Oleh karena itu, penelitian ini menekankan pengembangan *end-to-end* yang mencakup desain antarmuka, komponen *client-side* berbasis web yang mudah diakses, layanan *server-side*, mekanisme *data pipeline*, hingga pengukuran *user acceptance rate*. Secara garis besar, pengembangan produk *end-to-end* yang diajukan mencakup beberapa aspek berikut:

1. Analisis dan pengembangan API untuk memanggil dan meneruskan hasil inferensi model PFIRec sebagai model *issue recommender*.
2. Desain antarmuka pengguna yang konsisten dengan *design system* Primer oleh Github serta implementasi *client-side* dengan standar web modern.

3. Pengembangan komponen *server-side* dan *data pipeline* yang mampu mendukung pengoperasian berkelanjutan.
4. Pengujian teknis melalui *unit testing* serta *end-to-end testing* untuk memastikan ketahanan dan ketepatan sistem.
5. Evaluasi *user acceptance rate* melalui *task-based usability testing* untuk menilai efektivitas solusi dan mengidentifikasi peluang pengembangan lebih lanjut dari pengguna riil dengan latar belakang yang bervariasi.

Dengan demikian, penelitian ini tidak hanya menguji validitas model PFIRec dari sisi performa, tetapi juga menilai kematangannya sebagai produk *machine learning* yang siap digunakan dalam proses kontribusi perangkat proyek OSS secara nyata.

3.2. Metodologi

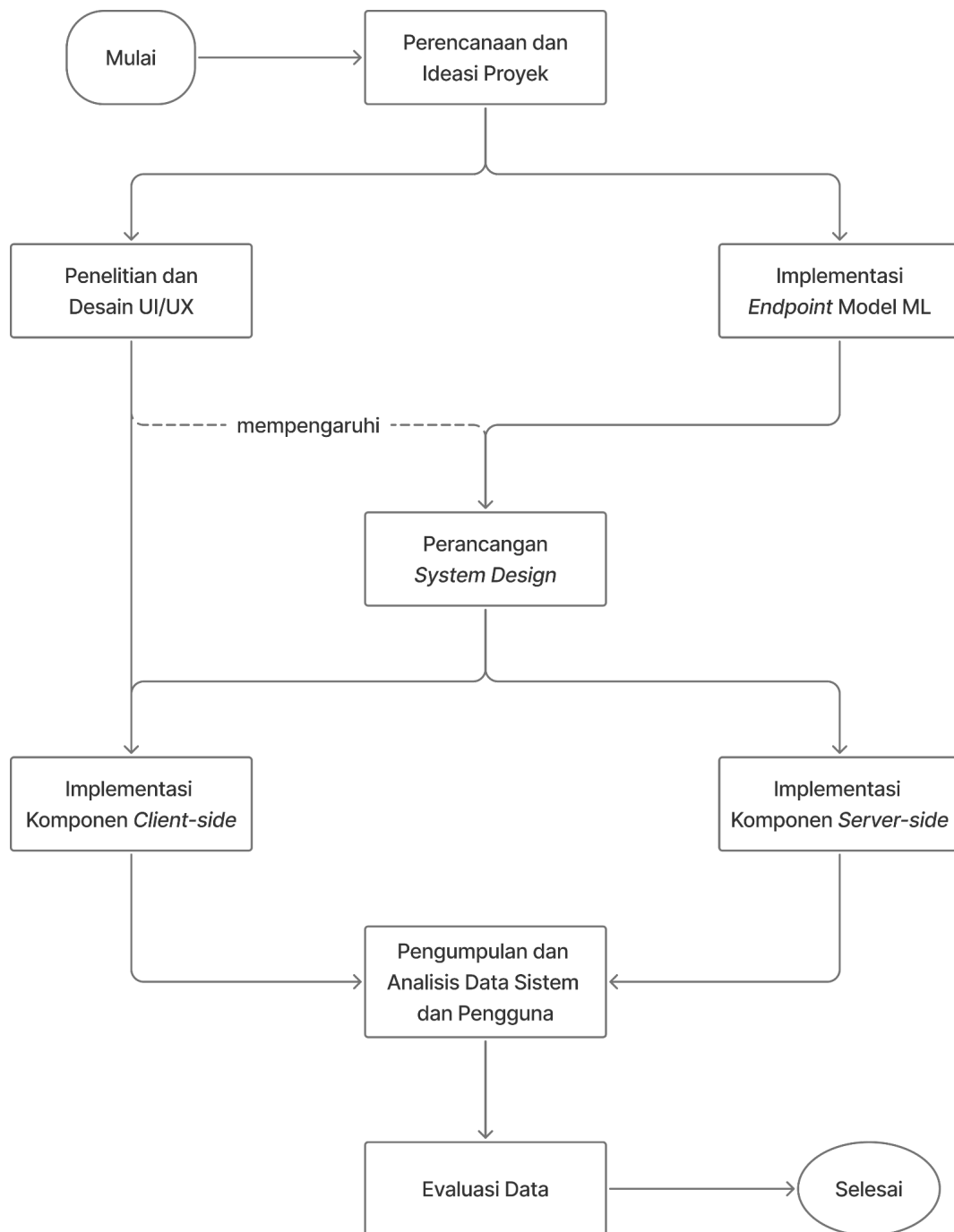
Kerangka penelitian dimulai dari tahap perencanaan yang memuat proses ideasi produk dan batasannya. Tahapan selanjutnya adalah tahap implementasi model *machine learning* GFI recommender yaitu PFIRec dari bentuk teoritis menjadi *endpoint* yang dapat menerima masukan melalui protokol HTTP. Secara paralel, melanjutkan dari tahapan ideasi, didefinisikan pula desain antarmuka aplikasi (UI/UX) yang mengakomodasi fitur-fitur yang telah direncanakan.

Memasuki fase pengembangan sistem, tahapan selanjutnya adalah perancangan arsitektur (*system design*) yang bertujuan untuk mendefinisikan komponen-komponen yang diperlukan dan tanggung jawabnya serta memetakan relasi dan aliran data di sistem. Berdasarkan arsitektur yang telah direncanakan, tahapan selanjutnya adalah pengembangan paralel di bagian *client-side* / *front-end* dan bagian *server-side* / *back-end*.

Fase terakhir dari penelitian yang diajukan adalah tahap pengumpulan dan evaluasi data sistem serta perilaku pengguna. Data terkait sistem dapat diperoleh dari hasil pengujian unit dan pengujian *end-to-end* melalui *system logging*. Sedangkan data terkait perilaku pengguna dapat diperoleh melalui proses *user testing* dan distribusi kuesioner. Data yang dikumpulkan dari dua tahapan ini

disintesis menjadi kesimpulan yang memvalidasi fungsionalitas dan tujuan utama aplikasi, serta menjadi acuan pengembangan sistem di masa mendatang.

Secara umum, penelitian ini akan dilakukan dengan beberapa tahapan seperti yang divisualisasikan pada Gambar 3.1.



Gambar 3.1. Bagan Alir Metodologi Penelitian

3.2.1. Tahap Perencanaan

Tahap paling awal dari implementasi sistem *end-to-end* untuk model PFIRec adalah mendefinisikan ruang lingkup serta kebutuhan sistem. Tahapan ini

diperlukan untuk mengarahkan fokus pengembangan produk inovasi pada kebutuhan yang esensial dan realistis. Secara umum, *Minimum Viable Product* (MVP) dari sistem *end-to-end* untuk model PFIRec dapat dikembangkan, namun tidak terbatas pada fitur-fitur berikut:

1. Autentikasi pengguna melalui GitHub OAuth untuk mendapatkan akses ke data profil pengguna, riwayat kontribusi, dan daftar *repository*.
2. Penerimaan masukan berupa tautan *repository* GitHub dari pengguna.
3. Ekstraksi data relevan dari *repository* yang diberikan (misalnya metadata *issue*, *label*, dan riwayat kontribusi dari pengguna).
4. Pengiriman data yang telah diekstraksi ke model PFIRec dalam bentuk yang telah ditentukan.
5. Penyimpanan dan visualisasi hasil rekomendasi *issue*.

Fitur-fitur terkait dapat menjadi basis awal yang bersifat minimal namun representatif pada alur kerja aplikasi, sehingga dapat digunakan sebagai dasar evaluasi dan penarikan data mengenai efektivitas implementasi model PFIRec. Adapun, luaran dari tahap ini adalah penjabaran *technical requirements* dan estimasi frekuensi penggunaan sistem oleh calon *contributor*.

3.2.2. Tahap Implementasi Model *Machine Learning*

Riset yang mengusulkan model PFIRec (Xiao dkk., 2023) membagikan *replication package* yang berisi berkas kode dari model PFIRec serta kode untuk memperoleh hasil analisis yang dijabarkan dalam penelitian. Selain itu, *replication package* disertai dengan kumpulan dataset terkait profil *contributor* dan *metadata issue* yang digunakan untuk proses *training* model. Dengan demikian, dapat dipahami bahwa *replication package* masih bersifat eksperimental dan tidak menyediakan berkas kode eksplisit untuk menangani proses ekstraksi data ataupun API agar model dapat diakses secara eksternal.

Oleh sebab itu, diperlukan tahapan khusus untuk memperluas implementasi model PFIRec ke dalam lingkungan tingkat industri. Langkah pertama adalah analisis implementasi model PFIRec, terutama pada bagian ekstraksi data, pra-pemrosesan data, hingga proses inferensi melalui algoritma

ranking LambdaMART. Setelah itu, diperlukan proses *training* model PFIRec melalui berkas kode yang disediakan agar dihasilkan berkas model PFIRec yang *modular*. Langkah terakhir di tahap implementasi model PFIRec adalah pengembangan API (*Application Programming Interface*) guna membuka akses programatik pada fungsi-fungsi inferensi pada model. Implementasi API paling efisien jika menggunakan bahasa pemrograman Python karena kompatibilitasnya yang tinggi dengan model PFIRec.

Luaran dari tahap ini adalah implementasi model PFIRec yang telah diperluas menjadi API. API terkait akan digunakan oleh komponen lain seperti komponen *server-side* sehingga dapat menerima data riil yang dikirim oleh pengguna, namun telah diekstraksi menjadi bentuk yang telah dirancang.

3.2.3. Tahap Implementasi Desain UI/UX

Dalam konteks produk yang diajukan, model PFIRec telah ditetapkan sebagai komponen inti sistem yang akan dikembangkan melalui pertimbangan teknis seperti akurasi model dan aspek inovatif model. Oleh karena itu, pendekatan *User-Centered Design* (UCD), sebagaimana telah diuraikan pada bagian sebelumnya, lebih difokuskan secara strategis pada perancangan desain antarmuka pengguna (UI/UX) yang optimal untuk model PFIRec. Tujuan utamanya adalah untuk memastikan bahwa fungsionalitas dan hasil keluaran dari model PFIRec dapat diakses, dipahami, dan dimanfaatkan secara efektif dan intuitif oleh calon pengguna, yakni komunitas *open-source*.

Dengan demikian, pendekatan UCD berperan krusial dalam menjembatani kapabilitas teknis model PFIRec dengan kebutuhan interaksi dan pengalaman pengguna akhir, guna memaksimalkan adopsi dan kegunaan sistem. Fokus utama dalam perancangan UI/UX dengan pendekatan UCD yang disesuaikan ini dapat dijabarkan sebagai berikut:

1. Desain Berbasis Tugas (*Task-Based*):

Mengidentifikasi fungsionalitas model PFIRec yang dapat diintegrasikan ke dalam alur kerja dari calon kontributor saat mencari dan memilih *issue*, dan merancang desain antarmuka yang mendukung alur tersebut.

2. **Minimasi Beban Kognitif:**

Menyajikan informasi dan rekomendasi dari model PFIRec dengan cara yang jelas, ringkas, dan menggunakan terminologi yang familiar bagi pengguna, guna memudahkan pemahaman dan pengambilan keputusan.

3. **Mekanisme Umpan Balik (*Feedback*):**

Mengantisipasi potensi waktu pemrosesan (ekstraksi data dan inferensi model) dengan menyediakan umpan balik visual dan *progress* yang jelas, sehingga sistem terasa responsif dan transparan.

Adapun proses desain UI/UX yang berpusat pada pengguna, dengan penyesuaian untuk model PFIRec yang telah ditentukan, akan melibatkan tahapan sistematis berikut:

1. **Analisis Kontekstual dan Komparatif pada Aplikasi Sejenis**

Melakukan analisis komparatif terhadap aplikasi sejenis, khususnya yang berkaitan dengan rekomendasi *issue* atau integrasi dengan platform seperti GitHub. Tujuannya adalah untuk memahami pola interaksi yang umum dan ekspektasi pengguna dalam ekosistem tersebut.

2. **Pemahaman Pengguna dalam Konteks PFIRec**

- Mengembangkan *user persona* yang merepresentasikan target pengguna PFIRec.
- Melakukan *user journey mapping* untuk memetakan bagaimana pengguna akan berinteraksi dengan sistem PFIRec dalam alur kerja mereka mencari kontribusi *open-source*.
- Memvalidasi hasil analisis perilaku pengguna melalui wawancara singkat atau informal dengan berbagai variasi demografi *software engineer* mulai dari tingkat mahasiswa hingga praktisi profesional.

3. **Ideasi Desain Antarmuka**

Mengeksplorasi berbagai alternatif desain antarmuka (UI) untuk menyajikan rekomendasi PFIRec dan fungsionalitas pendukungnya, menggunakan metode seperti *Crazy 8s*.

4. **Pengembangan Prototipe**

Mengembangkan sketsa desain, *wireframe* hingga desain akhir (*mockup*) menggunakan perangkat lunak seperti Figma. Prototipe ini akan merepresentasikan cara interaksi dengan PFIRec.

Semua tahapan ini dilaksanakan dengan asumsi pengguna utama adalah kontributor dengan latar belakang teknis yang familiar dengan platform VCS (*Version Control System*) seperti GitHub. Luaran dari tahap desain UI/UX yang dipandu UCD ini adalah fondasi untuk pengembangan antarmuka yang responsif dan relevan untuk model PFIRec, yang selanjutnya dapat dievaluasi efektivitasnya melalui pengujian pengguna.

3.2.4. Tahap Perancangan System Design

Tahap perancangan *system design* merupakan aspek fundamental dalam perancangan *sistem end-to-end* PFIRec yang bertujuan untuk mendesain interaksi seluruh komponen sistem agar dapat beroperasi dengan efisien dan diskalakan seiring peningkatan frekuensi penggunaan aplikasi. *System design* secara umum mencakup seluruh komponen pendukung sistem yang diperlukan seperti *client-side (front-end)*, *server-side (back-end)*, *database*, hingga *monitoring* dan *logging* dengan memperhatikan sumber daya yang tersedia.

Model PFIRec memiliki karakteristik yang perlu diperhatikan dari sisi *system design* yaitu proses ekstraksi data yang kompleks dan memakan waktu, inferensi model yang relatif cepat, serta kebutuhan untuk menyajikan hasil rekomendasi secara *near-real-time* kepada pengguna. Bagian yang dinilai menjadi tantangan utama adalah proses penyusunan dataset model yang melibatkan pengambilan puluhan fitur dari data GitHub API (metadata *issue*, profil pengguna, dan profil penulis *issue*) dan puluhan fitur yang perlu dikomputasi dari data sebelumnya. Pengambilan data melalui Github API berisiko memboroskan sumber daya jika tidak dilakukan penyimpanan data ataupun *caching* yang efisien, karena data yang sama dapat diambil berulang kali dan dikomputasi ulang.

Untuk kebutuhan pengembangan *Minimum Viable Product* (MVP), diasumsikan bahwa sistem akan melayani sekitar 20-30 pengguna aktif harian (DAU), dengan puncak *request* sebesar 10-20 permintaan per jam. Setiap

permintaan diasumsikan menghasilkan sekitar 120 fitur dari 3 sumber utama (pengguna, repositori, dan publisher issue), dengan ukuran rata-rata data sekitar 100 KB - 2 MB per permintaan setelah diproses. Artinya, sistem harus mampu menangani setidaknya 40 MB volume data per jam pada kondisi puncak, dengan potensi meningkat drastis jika digunakan pada proyek-proyek populer dengan ratusan *issue* aktif. Oleh karena itu, sistem perlu dirancang agar bersifat elastis, mampu menyimpan hasil ekstraksi untuk menghindari komputasi ulang, dan tetap responsif dalam menyajikan hasil inferensi model.

Rancangan arsitektur awal dari sistem *end-to-end* model PFIRec dibagi menjadi beberapa komponen utama:

1. *Client-side*

Aplikasi antarmuka yang bertugas untuk menangani autentikasi pengguna (melalui GitHub OAuth), menerima masukan berupa tautan *repository*, serta menampilkan hasil rekomendasi dari *back-end service*.

2. *Back-end Service (API Gateway)*

Layanan *stateless* yang berperan dalam menerima input kiriman dari *client-side*, memvalidasi *request*, mengirim *request* ke sistem *service worker* dan mengembalikan respon yang sesuai.

3. *Service Worker*

Service worker mengacu pada kumpulan komponen *back-end service* yang spesifik menangani pemrosesan asinkron secara terisolasi dan paralel agar tidak menghambat proses utama aplikasi. Adapun *request* yang dikirim ke satu *service worker* diatur dengan efisien melalui sistem antrian (*task queue*) seperti BullMQ (NodeJS), Async (Golang) dan Redis Queue. Dalam konteks proyek yang diajukan, *service worker* akan bertanggung jawab mengambil *task request* dari antrian, menjalankan proses ekstraksi data dari GitHub API, menghitung fitur-fitur yang diperlukan, dan menyimpan hasilnya ke dalam *database*. Ketika terdapat lonjakan penggunaan aplikasi, kumpulan (*cluster*) dari *service worker* dirancang untuk direplikasi sampai batasan tertentu (*scale up*) secara otomatis melalui konfigurasi Docker ataupun Kubernetes.

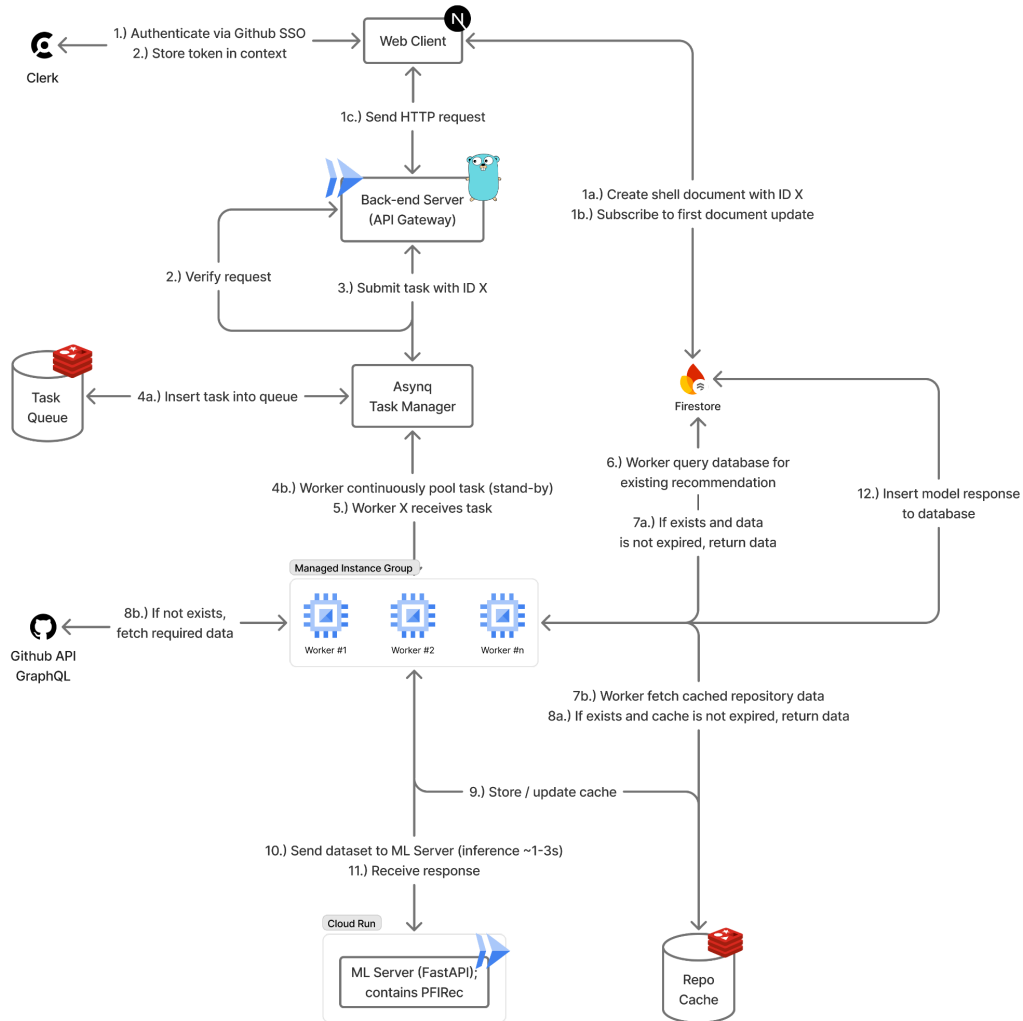
4. Database

Basis data yang digunakan untuk menyimpan hasil ekstraksi adalah Redis Sentinel karena sifat data *cache* yang perlu terbaharui. Sedangkan, basis data untuk menyimpan hasil inferensi (*issue recommendation*) adalah basis data sistem NoSQL seperti MongoDB atau Firebase Firestore, mengingat fleksibilitas skema dan kebutuhan akan penyimpanan dokumen yang kompleks.

5. Observabilitas

Kebutuhan observabilitas pada sistem *end-to-end* model PFIRec diperlukan untuk mengetahui kondisi operasional sistem secara *realtime*. Aspek observabilitas yang dibutuhkan mencakup *monitoring* performa dan status sistem, *logging* atau catatan proses yang berjalan, hingga beberapa metrik khusus yang mengukur frekuensi penggunaan fitur tertentu. Secara umum, kebutuhan observabilitas dapat dicapai dengan melibatkan komponen *monitoring* dan *logging* seperti Prometheus untuk metrik, Grafana untuk visualisasi, serta Loki untuk *log aggregation*.

Alur kerja logika bisnis aplikasi dan detail interaksi antarkomponen dari sistem *end-to-end* yang diajukan dapat dirujuk pada Gambar 3.2.



Gambar 3.2. Diagram *System Design*

Dengan pendekatan ini, luaran dari perencanaan *system design* adalah rancangan dari arsitektur *background worker* yang dapat diskalakan. Arsitektur ini diharapkan dapat menjadi dasar kuat bagi tahapan pengembangan dan evaluasi selanjutnya.

3.2.5. Tahap Implementasi Komponen *Server-side*

Sebagai perluasan dari *high-level architecture* yang dijabarkan dalam tahap perancangan *system design*, tahap selanjutnya difokuskan untuk rencana implementasi teknis komponen *server-side*. Secara umum, komponen *server-side* berperan sebagai penghubung *request* pengguna, proses ekstraksi data, dan

pemodelan *machine learning*. Oleh karena itu, perencanaan pengembangan pada komponen ini menjadi krusial dalam keberhasilan sistem secara menyeluruh.

Pendekatan rekayasa perangkat lunak yang diadopsi dalam pengembangan komponen *server-side* bersifat iteratif dan berorientasi pada keberlanjutan proyek. Pengembangan dilakukan dengan pendekatan *trunk-based development*, yaitu seluruh hasil pengembangan dilakukan pada satu cabang utama (main branch) Git dengan perubahan berskala kecil namun teruji. Adapun, setiap perubahan disertai dengan *unit test* dan *integration test* sesuai konteksnya. Untuk pengujian yang melibatkan *dependency* eksternal seperti GitHub API, strategi *mocking* diterapkan guna menjaga konsistensi *behavior* aplikasi. Pendekatan ini diharapkan dapat menghasilkan berkas kode proyek yang bersih, modular dan mendukung *maintainability*.

Komponen *server-side* dalam konteks sistem yang diajukan secara umum terbagi menjadi dua jenis layanan:

1. API gateway yang menjadi penghubung langsung dengan sisi klien untuk menerima input, autentikasi pengguna, serta menyajikan hasil rekomendasi.
2. *Service worker* yang berfungsi menjalankan proses ekstraksi data dan komputasi fitur secara asinkron, sehingga beban kerja yang berat tidak berdampak langsung terhadap waktu respons layanan utama.

Layanan API gateway diimplementasikan dengan protokol REST API dan menggunakan bahasa Go dengan framework HTTP ringan seperti Hertz yang menawarkan performa tinggi. Layanan ini mencakup *endpoint* untuk autentikasi OAuth, serta pengiriman *request*. Selain itu, dilakukan pula operasi validasi *input*, penanganan galat, dan pengiriman *request* ke komponen *task queue*. Untuk mengantisipasi galat akibat *rate limit* dari GitHub API, layanan ini juga dilengkapi dengan mekanisme *timeout* dan *fallback* yang sesuai.

Sementara itu, layanan *service worker* dirancang untuk menangani beban kerja intensif secara paralel. Implementasi dilakukan dengan menggunakan *tool task queue* berbasis Redis seperti Asynq yang mendukung operasi penjadwalan dan distribusi *task*. Setiap *task* yang diterima akan diproses secara *concurrent*

dengan memanfaatkan fitur *goroutine* dalam bahasa Go. Tanggung jawab utama dari suatu *instance service worker* adalah mengekstraksi dan pemrosesan data sesuai *request* melalui Github API GraphQL, yang dipilih karena efisiensinya. Selain itu, *service worker* juga perlu menyimpan hasil ekstraksi data ke dan komputasi terkait ke dalam penyimpanan sementara seperti *cache* di Redis. Adapun, data hasil inferensi dari model PFIRec dapat disimpan sebagai riwayat maupun *long-lived cache* dalam *database* utama seperti Firebase Firestore (NoSQL).

Dengan pendekatan ini, sistem *server-side* memiliki dua layanan yang terpisah dengan tugas dan karakteristik yang spesifik. Pemisahan tanggung jawab ini tidak hanya mendukung skalabilitas sistem, tetapi juga memungkinkan proses pengembangan dan optimisasi dilakukan secara modular.

3.2.6. Tahap Implementasi Komponen *Client-side*

Komponen *client-side* merupakan perluasan dari *high-level architecture* yang berperan sebagai antarmuka utama antara pengguna dan sistem rekomendasi. Dalam konteks sistem *end-to-end* dari model PFIRec, sistem ini dirancang *web application*. Pemilihan bentuk *web application* didasari oleh alasan aksesibilitas, kemudahan penggunaan, serta familiaritas target pengguna yang merupakan *contributor* proyek OSS terhadap sistem yang serupa seperti GitHub dan GitLab. Sebagai aplikasi berbasis web, pengguna tidak perlu melakukan instalasi dan dapat langsung mengakses aplikasi melalui perangkat apa pun.

Secara umum, tanggung jawab dari komponen *client-side* dapat dijabarkan sebagai berikut:

1. Menangani autentikasi melalui Github SSO, dimana *token* yang dikembalikan akan digunakan sebagai *token* autentikasi saat pemanggilan Github API, terutama pada *private repository*.
2. Menerima masukan dari pengguna berupa tautan *repository* dan ID pengguna yang sebelumnya telah didapatkan melalui proses autentikasi.

3. Mengeksekusi proses *reactive subscription* dengan membuat entri dokumen dengan ID yang telah ditentukan di database utama, lalu mengirim *request* dengan data terkait ke *API Gateway*.
4. Memvisualisasikan hasil inferensi ketika dokumen *database* yang terkait telah diperbarui untuk yang pertama kali.

Dari segi pendekatan *software engineering*, pengembangan komponen *client-side* akan menggunakan arsitektur berbasis komponen serta trunk-based development untuk mendorong iterasi yang cepat dan terintegrasi. Fokus pengembangan diletakkan pada akurasi antarmuka sesuai desain UI/UX, termasuk sisi responsivitas antarmuka, dan petunjuk visual yang reflektif terhadap kondisi di sisi *server-side*.

Implementasi komponen *client-side* direncanakan menggunakan *framework* NextJS karena dukungan fitur *server-side rendering* (SSR) yang dapat mengurangi waktu memuat halaman, manajemen rute halaman yang terstruktur, hingga fleksibilitas dalam pengelolaan API. Untuk pengembangan antarmuka visual, akan digunakan *tool* Tailwind CSS karena pendekatan *utility-first* yang memungkinkan pengembangan UI secara efisien dan konsisten. Terakhir, *deployment* aplikasi akan menggunakan layanan Vercel, yang menyediakan integrasi langsung dengan GitHub untuk CI/CD serta kemampuan optimasi performa berbasis CDN.

3.2.7. Tahap Pengumpulan dan Analisis Data Performa Sistem serta Perilaku Pengguna

Tahap selanjutnya setelah tahap implementasi menyeluruh pada sistem *end-to-end* model PFIRec adalah tahap evaluasi. Tahap evaluasi yang objektif memerlukan substansi data yang akurat, bervariasi, dan relevan terhadap kasus yang akan diuji. Maka, diperlukan metode yang sistematis dalam proses penentuan data-data yang relevan, detail prosedur pengumpulan data terkait hingga metode analisis data yang sesuai.

Aspek pertama yang akan dianalisis adalah performa dan fungsionalitas sistem *end-to-end* untuk model PFIRec. Pendekatan teknis yang umum digunakan dijabarkan sebagai berikut:

1. Pengujian Unit (*Unit Testing*)

Pengujian unit atau *unit testing* bertujuan untuk menguji fungsionalitas setiap komponen sistem, baik di bagian *client-side* ataupun *server-side* secara individu dan tidak terikat. Dengan demikian, setiap fungsionalitas dari komponen dapat diverifikasi. Sebagai contoh, komponen autentikasi di *web app (client-side)* dapat diuji apakah dapat ditekan oleh pengguna, menampilkan visualisasi status yang tepat, hingga *edge cases* seperti menampilkan galat yang sesuai.

Khusus dalam konteks *client-side*, pengujian hanya akan dilakukan secara selektif menggunakan *tool* seperti React Testing Library jika diperlukan, mengingat sebagian besar logika di aplikasi *client-side* bersifat presentasional. Adapun, setiap *framework* dan bahasa pemrograman yang diajukan sebagai dasar implementasi seperti Javascript ataupun Golang memiliki fitur pengujian yang komprehensif.

2. Pengujian *End-to-End* (*End-to-End Testing*)

Setelah pengujian unit dari sistem mencapai sekitar 80% cakupan (*coverage*), pengumpulan data sistem akan ditindaklanjuti dengan pengujian *end-to-end*. Pengujian ini melibatkan seluruh komponen di sistem dan dijalankan berdasarkan skenario yang menyerupai aksi yang mungkin dilakukan oleh pengguna. Secara umum, skenario pengujian *end-to-end* dapat dijabarkan sebagai berikut:

- Proses pendaftaran pengguna yang melibatkan *web app (client-side)* di halaman autentikasi, dan kode *business logic* ke layanan pihak ketiga: Clerk.
- Proses *onboarding* dan pembentukan profil pengguna yang melibatkan *web app (client-side)* di halaman *onboarding* yang menerima masukan profil dan preferensi *domain* pengguna,

komponen *back-end* di *endpoint onboarding*, serta koneksi ke *database* Firestore.

- Proses rekomendasi *issue* yang merupakan fitur utama aplikasi dan melibatkan seluruh komponen *client-side*, *back-end*, *service worker*, dan AI model.

Aspek kedua yang akan dianalisis adalah perilaku pengguna dengan pendekatan yang diajukan mengacu pada prinsip *user-centered design*. Pendekatan ini konsisten dengan metode yang digunakan pada tahap *ideation* di dalam konteks desain UI/UX, namun pada tahap pengumpulan data dan evaluasi, konsep UCD lebih mengacu pada evaluasi penerimaan pengguna (*user acceptance rate*) yang bertujuan untuk menentukan:

1. Apakah sistem *end-to-end* model PFIRec efektif dalam membantu pengguna untuk menemukan *issue* yang relevan?
2. Bagaimana proses pengguna berinteraksi dengan sistem?
3. Bagaimana bentuk tantangan dan friksi yang dialami pengguna dalam menggunakan sistem?

Tujuan akhir yang telah dijabarkan di atas akan disimpulkan dalam tahap berikutnya yaitu bagian evaluasi sistem dan data. Dalam ranah desain produk (*product design*), pendekatan yang umum digunakan untuk mengumpulkan data perilaku (*behavior*) pengguna adalah melalui proses *user testing*, *interview* hingga kuesioner sederhana. Untuk mengumpulkan data yang diperlukan secara sistematis, diajukan pendekatan sebagai berikut:

1. Task-Based Usability Test

Task-based usability testing adalah metode evaluasi yang menekankan pada pengamatan langsung terhadap cara pengguna menyelesaikan sejumlah tugas yang telah didefinisikan sebelumnya, dalam konteks penggunaan sistem *end-to-end* model PFIRec. Metode ini merupakan turunan dari pendekatan *user testing* konvensional, namun dengan fokus eksplisit pada penyelesaian tugas yang merepresentasikan skenario penggunaan nyata.

Melalui metode ini, sistem akan diuji oleh partisipan dengan instruksi tugas tertentu, seperti: “Temukan isu publik yang relevan berdasarkan minat atau preferensi pribadi,” atau “Temukan riwayat rekomendasi isu yang telah Anda minta sebelumnya”.

Metode ini memungkinkan pengumpulan data kuantitatif, seperti *task success rate* (tingkat keberhasilan tugas), *time on task* (durasi penyelesaian), dan *click path* (alur navigasi pengguna). Selain itu, metode ini juga dapat mengumpulkan data kualitatif melalui metode *think-aloud*, dimana partisipan diminta untuk mengungkapkan pemikirannya secara verbal selama proses interaksi.

Dalam eksekusinya, *task-based usability testing* dapat difasilitasi melalui wawancara langsung oleh peneliti atau melalui *platform* observasi daring seperti *usertesting.com*. Untuk memperluas dan memperkuat hasil temuan data, peneliti merencanakan untuk mewawancarai 40-50 *software engineer* dari berbagai kelompok demografi seperti mahasiswa tingkat awal, mahasiswa tingkat akhir, praktisi akademik hingga praktisi industri baik secara langsung ataupun daring. Secara umum, hasil dari proses ini akan memberikan gambaran komprehensif dan variatif mengenai perilaku pengguna saat menggunakan sistem.

2. Distribusi Kuesioner

Kuesioner dirancang sebagai instrumen pelengkap yang dikirimkan tepat setelah tahap *task-based usability testing* untuk menangkap persepsi, preferensi, dan tingkat penerimaan pengguna terhadap sistem secara eksplisit. Metode ini bersifat reflektif dan subjektif dan bertujuan memberikan ruang bagi pengguna untuk mengevaluasi pengalaman mereka secara pribadi setelah berinteraksi dengan sistem.

Rencana distribusi kuesioner dilakukan dalam bentuk digital agar lebih mudah diakses partisipan dan akan dibagikan langsung setelah proses wawancara *Task-Based Usability Testing* selesai dilaksanakan. Adapun, kumpulan pertanyaan yang diajukan dapat mencakup dimensi seperti:

- Kemudahan penggunaan sistem (*usability*),

- Kejelasan dan relevansi hasil rekomendasi *issue*,
- Kepercayaan terhadap sistem dalam menyarankan isu publik yang relevan.
- Saran terbuka bagi pengembangan sistem di masa mendatang

Data yang dihasilkan dari kuesioner ini bersifat kualitatif dan kuantitatif, termasuk nilai skala (misal 1 - 5) dan pertanyaan terbuka. Informasi ini akan digunakan untuk melengkapi data observasi dan memberikan validasi dari sisi persepsi pengguna secara eksplisit.

3. *System Logging*

System logging adalah proses pencatatan otomatis terhadap aktivitas sistem dan interaksi pengguna dalam bentuk *log* yang terstruktur. Data ini dikumpulkan oleh sistem secara pasif selama sistem berjalan dan tidak memerlukan partisipasi eksplisit dari pengguna. Tujuan utama dari *logging* adalah untuk menyediakan data objektif yang berkaitan dengan performa sistem, tingkat efisiensi komputasi, serta perilaku aktual dari pengguna. *Log* dapat mencakup informasi-informasi seperti:

- Frekuensi dan waktu akses fitur tertentu
- Jumlah dan jenis permintaan rekomendasi,
- Durasi penggunaan sistem (sesi) dari tiap pengguna,
- Beban kerja yang dikonsumsi oleh masing-masing komponen (terutama pada komponen *task queue* dan *service worker*),
- Performa proses inferensi dari model (latensi, tingkat keberhasilan, maupun tingkat *retry*).

Data yang dikumpulkan melalui *system logging* akan digunakan dalam evaluasi fungsionalitas sistem. Selain itu, data perilaku pengguna juga dapat diekstraksi untuk menyempurnakan pemahaman akan perilaku dan preferensi pengguna dari proses *task-based usability test*.

3.2.8. Tahap Evaluasi

Tahap evaluasi bertujuan untuk mengukur keberhasilan implementasi sistem *end-to-end* model PFIRec berdasarkan indikator

user-acceptance dan variasi *behavior* dari pihak pengguna. Evaluasi dilakukan secara holistik terhadap berbagai dimensi seperti pengalaman pengguna (*usability*), akurasi rekomendasi, efisiensi sistem dalam konteks beban kerja dan latensi, hingga potensi skalabilitas sistem. Seluruh pengukuran terkait didasarkan pada data yang terkumpul dan indikator keberhasilan yang dapat dijabarkan sebagai berikut:

1. Task-Based Usability Test

- *Task success rate*: $\geq 80\%$ partisipan menyelesaikan tugas tanpa bantuan mengindikasikan antarmuka sistem cukup intuitif.
- *Time on task*: Durasi penyelesaian tugas berada dalam rentang wajar ($\pm 20\%$ dari waktu ideal yang akan ditentukan)
- *Click path accuracy*: Mayoritas partisipan mengikuti alur penggunaan yang telah dirancang tanpa penyimpangan yang signifikan
- *Think-aloud feedback*: Mayoritas *feedback* dan komentar verbal bersifat positif, menunjukkan pemahaman, dan tidak mengindikasikan perilaku kebingungan, ataupun frustrasi terhadap mekanisme sistem.

2. Kuesioner

- Pertanyaan dengan model skor 1–5 menunjukkan skor rata-rata ≥ 3.5 pada aspek kemudahan penggunaan dan relevansi hasil rekomendasi.
- Tanggapan terbuka mencerminkan bahwa sistem dianggap bermanfaat, dan mudah digunakan.

3. System Logging

- Waktu inferensi model < 5 detik pada beban kerja rata-rata yang mengindikasikan implementasi model cukup optimal dan secara umum menghasilkan latensi yang dapat diterima pengguna.
- Tingkat *retry* dari model $< 5\%$ pada beban kerja rata-rata menunjukkan tingkat kegagalan atau ketidakakuratan model yang cukup rendah.
- Distribusi *task queue* pada *service worker* merata sehingga mengindikasikan efisiensi sistem dan ketiadaan *bottleneck*.

BAB IV

JADWAL PENELITIAN

4.1. Jadwal Penelitian

Penelitian ini dilakukan dengan memperhatikan waktu pengerjaan sesuai dengan tahapan yang dijabarkan dalam metodologi riset, indikator penelitian serta alur pelaksanaan tugas akhir sedemikian dijabarkan oleh program studi Ilmu Komputer Fakultas Matematika dan Ilmu Pengetahuan Alam (FMIPA), Universitas Gadjah Mada. Adapun, estimasi jadwal dari tiap tahap dijabarkan melalui Tabel 4.1. dan Tabel 4.2.

	April 2025				Mei 2025				Juni 2025				Juli 2025			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Persiapan																
Tinjauan Pustaka																
Analisis Model																
Penyusunan Proposal																
Pengujian Proposal																
Implementasi																
Implementasi <i>Endpoint</i> Model																
Ideasi Produk																
Penelitian dan Desain UI/UX																
Perancangan Arsitektur Sistem																
Implementasi Infrastruktur Sistem																

Tabel 4.1. Jadwal Penelitian dari Bulan Satu hingga Empat

	Agustus 2025				September 2025				Oktober 2025				November 2025			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Implementasi Aplikasi <i>Server-Side</i>																
Implementasi Aplikasi <i>Client-Side</i>																
Implementasi Sistem <i>End-to-End</i>																
Pengujian dan Perilisan Sistem																
Seminar Skripsi																
Evaluasi																
Pengumpulan Data																
Evaluasi Data																
Laporan Akhir																
Penyusunan Laporan Akhir																
Pengujian Laporan Akhir																

Tabel 4.1. Jadwal Penelitian dari Bulan Lima hingga Delapan

GLOSARIUM

Kontrol Versi dan Kolaborasi

- **Git**: Sistem kontrol versi terdistribusi untuk melacak perubahan kode dan memfasilitasi kolaborasi pengembangan perangkat lunak.
- **VCS (Version Control System)**: Sistem yang mengelola perubahan pada kumpulan *file* dari waktu ke waktu, memungkinkan pelacakan riwayat dan kolaborasi.
- **GitHub**: Platform *hosting repository* Git berbasis *web* yang memfasilitasi kolaborasi pengembangan perangkat lunak dan manajemen proyek.
- **Issue**: Mekanisme di platform pengembangan perangkat lunak (misalnya GitHub) untuk melacak *bug*, permintaan fitur, atau tugas yang perlu dikerjakan.
- **Good First Issue (GFI)**: Label pada *issue* proyek *open-source* yang menandakan tugas tersebut cocok untuk kontributor baru karena kompleksitasnya yang rendah.
- **Kontributor**: Individu yang secara sukarela berkontribusi pada pengembangan atau pemeliharaan proyek *open-source*.
- **Core Maintainer**: Anggota tim inti proyek *open-source* yang bertanggung jawab atas arah, kualitas kode, dan manajemen komunitas.
- **Onboarding**: Proses pengenalan dan integrasi kontributor atau pengguna baru ke dalam suatu proyek atau sistem, termasuk pemahaman alur kerja dan budaya.

Arsitektur dan Pola Pengembangan Perangkat Lunak

- **Queue**: Struktur data atau mekanisme yang menyimpan daftar tugas yang menunggu untuk diproses secara berurutan.
- **Background Worker**: Komponen sistem yang memproses tugas-tugas berat atau asinkron di latar belakang tanpa mengganggu responsivitas aplikasi utama.
- **API (Application Programming Interface)**: Kumpulan definisi protokol yang memungkinkan aplikasi untuk berkomunikasi satu sama lain.

- **REST (Representational State Transfer):** Gaya arsitektur untuk sistem *hypermedia* terdistribusi, sering digunakan untuk membangun *web API*.
- **GraphQL:** Bahasa kueri dan *runtime* sisi *server* untuk *API* yang memungkinkan klien meminta data yang tepat yang mereka butuhkan.
- **End-to-End (dalam konteks proyek ini):** Pendekatan pengembangan sistem yang mencakup seluruh alur dari input pengguna hingga output akhir, termasuk *frontend*, *backend*, dan integrasi model *AI/ML*.
- **Client-side:** Bagian aplikasi yang berjalan di perangkat pengguna (misalnya *web browser* atau aplikasi seluler).
- **Server-side:** Bagian aplikasi yang berjalan di *server* dan bertanggung jawab untuk pemrosesan data, logika bisnis, dan akses ke *database*.
- **API Gateway:** Titik masuk tunggal untuk semua klien yang mengakses layanan *backend*, menangani keamanan dan manajemen lalu lintas.
- **MVP (Minimum Viable Product):** Versi produk baru dengan fitur yang cukup untuk memuaskan pelanggan awal dan memberikan umpan balik untuk pengembangan di masa mendatang.

Desain dan Pengalaman Pengguna

- **User-Centered Design (UCD):** Metodologi desain yang menempatkan pengguna sebagai fokus utama di setiap tahapan pengembangan produk.
- **UI/UX:** Akronim untuk *User Interface* (antarmuka pengguna) dan *User Experience* (pengalaman pengguna).
- **UI/UX Design:** Disiplin yang berfokus pada perancangan antarmuka visual dan interaksi produk untuk menciptakan pengalaman pengguna yang optimal.
- **Crazy 8s:** Teknik *brainstorming* desain cepat di mana peserta menggambar delapan ide sketsa dalam delapan menit.

Keamanan

- **XSS (Cross-Site Scripting):** Jenis kerentanan keamanan *web* yang memungkinkan penyerang menyuntikkan *script* berbahaya ke dalam halaman *web* yang dilihat pengguna.

- **CSP (Content Security Policy):** Standar keamanan *web* yang membantu mencegah serangan XSS dan injeksi data lainnya dengan mengontrol sumber daya yang diizinkan untuk dimuat oleh *browser*.
- **OAuth (Open Authorization):** Standar terbuka untuk delegasi otorisasi, sering digunakan untuk memungkinkan aplikasi pihak ketiga mengakses informasi pengguna tanpa memberikan kredensial lengkap.

Operasi dan Pemantauan

- **Observabilitas:** Kemampuan suatu sistem untuk memahami kondisi internalnya berdasarkan data yang dihasilkan (log, metrik, *trace*).
- **MCP (Machine Context Protocol):** Protokol yang digunakan untuk pertukaran informasi konteks atau status antar mesin atau komponen dalam sistem.

DAFTAR PUSTAKA

- Asaad, C., Bařna, K., Ghogho, M., 2020. NoSQL Databases: Yearning for Disambiguation. <https://doi.org/10.48550/arXiv.2003.04074>
- Chammas, A., Quaresma, M., Mont'Alvão, C., 2015. A Closer Look on the User Centred Design. *Procedia Manuf.* 3, 5397–5404. <https://doi.org/10.1016/j.promfg.2015.07.656>
- Gomes, C., de O. Junior, M.N., Nogueira, B., Maciel, P., Tavares, E., 2023. NoSQL-based storage systems: influence of consistency on performance, availability and energy consumption. *J. Supercomput.* 79, 21424–21448. <https://doi.org/10.1007/s11227-023-05488-6>
- Haerder, T., Reuter, A., 1983. Principles of transaction-oriented database recovery. *ACM Comput. Surv.* 15, 287–317. <https://doi.org/10.1145/289.291>
- Harrison Oke Ekpobimi, 2024. Building high-performance web applications with NextJS. *Comput. Sci. IT Res. J.* 5, 1963–1977. <https://doi.org/10.51594/csitrj.v5i8.1459>
- Jin, R., Cordingly, R., Zhao, D., Lloyd, W., 2024. GraphQL vs. REST: A Performance and Cost Investigation for Serverless Applications, in: *Proceedings of the 10th International Workshop on Serverless Computing, WoSC10 '24*. Association for Computing Machinery, New York, NY, USA, pp. 37–42. <https://doi.org/10.1145/3702634.3702956>
- Qi Zhang, Ningfang Mi, Smirni, E., Riska, A., Riedel, E., 2006. Evaluating the Performability of Systems with Background Jobs, in: *International Conference on Dependable Systems and Networks (DSN'06)*. Presented at the International Conference on Dependable Systems and Networks (DSN'06), IEEE, Philadelphia, PA, USA, pp. 495–504. <https://doi.org/10.1109/DSN.2006.33>
- Schueller, W., Wachs, J., 2024. Modeling interconnected social and technical risks in open source software ecosystems. *Collect. Intell.* 3, 26339137241231912. <https://doi.org/10.1177/26339137241231912>
- Silberschatz, A., Korth, H.F., Sudarshan, S., 2011. *Database system concepts*, 6th ed. ed. McGraw-Hill, New York, NY.
- Steinmacher, I., Silva, M.A.G., Gerosa, M.A., 2014. Barriers Faced by Newcomers to Open Source Projects: A Systematic Review, in: Corral, L., Sillitti, A., Succi, G., Vlasenko, J., Wasserman, A.I. (Eds.), *Open Source Software: Mobile Open Source Technologies, IFIP Advances in Information and Communication Technology*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 153–163. https://doi.org/10.1007/978-3-642-55128-4_21
- Tan, X., Zhou, M., Sun, Z., 2020. A first look at good first issues on GitHub, in: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Presented at the ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ACM, Virtual Event USA, pp. 398–409.

- <https://doi.org/10.1145/3368089.3409746>
- Tsitsiklis, J.N., Xu, K., 2017. Flexible Queueing Architectures. *Oper. Res.* 65, 1398–1413. <https://doi.org/10.1287/opre.2017.1620>
- Vargovich, J., Santos, F., Penney, J., Gerosa, M.A., Steinmacher, I., 2023. GiveMeLabeledIssues: An Open Source Issue Recommendation System, in: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR). pp. 402–406. <https://doi.org/10.1109/MSR59073.2023.00061>
- Xiao, W., Li, J., He, H., Qiu, R., Zhou, M., 2023. Personalized First Issue Recommender for Newcomers in Open Source Projects. Presented at the The 38th IEEE/ACM International Conference on Automated Software Engineering (ASE 2023), arXiv. <https://doi.org/10.48550/arXiv.2308.09038>
- Xiaoshu, W., 2020. Optimized Development of Web Front-end Development Technology. *J. Phys. Conf. Ser.* 1693, 012057. <https://doi.org/10.1088/1742-6596/1693/1/012057>
- Zebec, S.T., 2016. GitHub open source project recommendation system. ArXiv.
- Zhou, M., Mockus, A., 2012. What make long term contributors: Willingness and opportunity in OSS community, in: 2012 34th International Conference on Software Engineering (ICSE). Presented at the 2012 34th International Conference on Software Engineering (ICSE 2012), IEEE, Zurich, pp. 518–528. <https://doi.org/10.1109/ICSE.2012.6227164>